

**LAPORAN PRAKTIKUM**  
**PRAKTIKUM PENAMBANGAN DATA**  
**PERTEMUAN 6**  
**CLUSTERING DAN ASSOCIATION RULE**



**Disusun oleh:**

**Nama** : Hafidz Rizqullah Prasetya  
**NIM** : 24/535493/SV/24243  
**Kelas** : PL5A1  
**Dosen Pengampu** : Dr. Imam Fahrurrozi, S.T., M.Cs.

**PROGRAM STUDI D-IV**  
**TEKNOLOGI REKAYASA PERANGKAT LUNAK**  
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**  
**SEKOLAH VOKASI**  
**UNIVERSITAS GADJAH MADA**  
**YOGYAKARTA**  
**2026**

# Daftar Isi

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Daftar Isi</b>                                                              | <b>2</b>  |
| <b>Tujuan Praktikum</b>                                                        | <b>3</b>  |
| <b>Dasar Teori</b>                                                             | <b>4</b>  |
| 2.1 Unsupervised Learning                                                      | 4         |
| 2.2 Konsep Dasar Klasterisasi                                                  | 5         |
| 2.3 Algoritma K-Means                                                          | 5         |
| 2.4 Evaluasi Kualitas Klaster dan Penentuan Nilai K                            | 8         |
| 2.4.1 Inertia (Within-Cluster Sum of Squares)                                  | 8         |
| 2.4.2 Silhouette Score                                                         | 8         |
| 2.4.3 Penentuan Nilai K Optimal                                                | 9         |
| 2.5 Algoritma K-Medoids (Partitioning Around Medoids)                          | 9         |
| 2.6 Algoritma DBSCAN                                                           | 11        |
| 2.7 Association Rule Mining                                                    | 13        |
| 2.8 Algoritma Apriori                                                          | 14        |
| <b>Alat dan Bahan</b>                                                          | <b>16</b> |
| <b>Hasil dan Pembahasan</b>                                                    | <b>17</b> |
| 4.1 Percobaan Latihan Praktikum                                                | 17        |
| 4.1.1 Persiapan dan Impor Pustaka                                              | 17        |
| 4.1.2 Membaca dan Memuat Informasi Dataset                                     | 18        |
| 4.1.3 Pembersihan Data (Data Cleaning)                                         | 20        |
| 4.1.4 Standarisasi Skala Fitur Data                                            | 22        |
| 4.1.5 Penentuan Nilai K Optimal pada K-Means                                   | 22        |
| 4.1.6 Penentuan Nilai K Optimal pada K-Medoids                                 | 25        |
| 4.1.7 Evaluasi Silhouette Score pada K-Means dan K-Medoids                     | 26        |
| 4.1.8 Pelatihan Model K-Means ( $K = 4$ ) dan Visualisasi                      | 28        |
| 4.1.9 Pelatihan Model K-Medoids ( $K = 4$ ) dan Visualisasi                    | 30        |
| 4.1.10 Analisis Profil Klaster dan Interpretasi Bisnis                         | 32        |
| 4.2 Tugas dan Analisis Praktikum                                               | 34        |
| 4.2.1 Soal 1: Dataset Preparation (Credit Card Dataset)                        | 34        |
| 4.2.1.1 Tujuan Penggunaan Dataset                                              | 34        |
| 4.2.1.2 Deskripsi dan Makna Fitur Dataset                                      | 34        |
| 4.2.2 Soal 2: Eksperimen Klasterisasi Dataset Air Traffic Passenger Statistics | 35        |
| 4.2.2.1 Pemrosesan Data dan Pengkodean Kategori                                | 35        |
| 4.2.2.2 Penentuan Nilai K Optimal pada Data Penerbangan                        | 36        |
| 4.2.2.3 Penerapan K-Means dan K-Medoids                                        | 37        |
| 4.2.2.4 Analisis Ketahanan terhadap Outlier                                    | 38        |
| 4.2.3 Soal 3: Verifikasi Komputasi Association Rule dan Prinsip Apriori        | 39        |
| <b>Kesimpulan</b>                                                              | <b>40</b> |
| <b>Daftar Pustaka</b>                                                          | <b>41</b> |

# Tujuan Praktikum

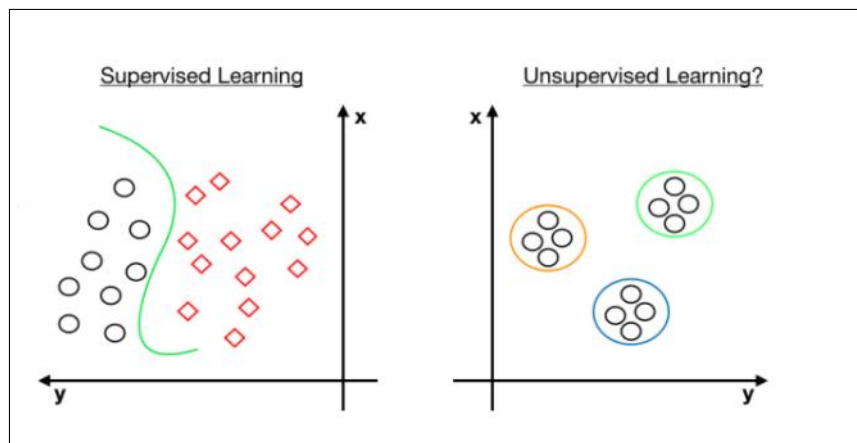
Berdasarkan Modul Pertemuan 6 Praktikum Penambangan Data, tujuan praktikum ini adalah:

1. Mahasiswa memahami konsep dasar unsupervised learning dan perbedaannya dengan supervised learning.
2. Mahasiswa memahami prinsip kerja, kelebihan, dan kelemahan algoritma klusterisasi K-Means, K-Medoids (Partitioning Around Medoids / PAM), dan DBSCAN (Density-Based Spatial Clustering of Applications with Noise).
3. Mahasiswa dapat mengevaluasi kualitas kluster memakai metrik evaluasi internal seperti Inertia (Within-Cluster Sum of Squares / WCSS) dan Silhouette Score, serta menentukan jumlah kluster optimal ( $K$ ) memakai Elbow Method dan KneeLocator.
4. Mahasiswa dapat menerapkan visualisasi hasil klusterisasi dalam dimensi dua memakai Matplotlib dan Seaborn, serta dimensi tiga memakai Plotly Express.
5. Mahasiswa mampu melakukan analisis profil kluster (cluster profiling) dan interpretasi bisnis atas segmen nasabah yang terbentuk.
6. Mahasiswa memahami konsep dasar Association Rule Mining, metrik evaluasi Support, Confidence, dan Lift, prinsip Apriori (anti-monotonicity of support), serta mekanisme pemangkasan kandidat itemset (lattice pruning).

# Dasar Teori

## 2.1 Unsupervised Learning

Unsupervised learning adalah paradigma pembelajaran mesin yang melatih model pada data tanpa label target ( $y$ ). Model bertugas mengenali pola keteraturan, kesamaan atribut, atau struktur kepadatan yang terbentuk secara alami dari matriks fitur masukan ( $X$ ) [1, 2]. Gambar 1 memperlihatkan perbedaan konsep kedua paradigma ini.



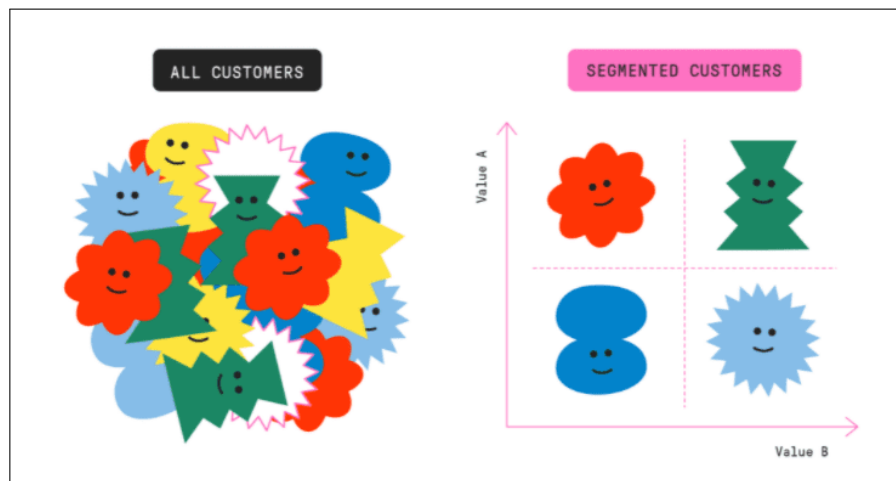
Gambar 1: Perbandingan pembelajaran mesin: supervised learning memakai data berlabel sebagai acuan target, sedangkan unsupervised learning mengelompokkan data tanpa label [12].

Perbedaan pokok kedua pendekatan ini meliputi:

- **Supervised Learning:** Memetakan hubungan fungsi matematis  $f(X) \approx y$  antara pasangan masukan dan target yang telah diketahui. Tujuannya adalah memprediksi nilai target pada sampel baru melalui tugas klasifikasi atau regresi.
- **Unsupervised Learning:** Hanya menerima variabel fitur masukan  $X$ . Tujuannya adalah mengidentifikasi struktur intrinsik data, seperti pengelompokan (clustering), penemuan aturan asosiasi (association rules), atau reduksi dimensi data.

Penerapan unsupervised learning di industri mencakup segmentasi pelanggan untuk mengelompokkan nasabah berdasarkan pola transaksi, deteksi anomali untuk mendeteksi

transaksi kartu kredit mencurigakan, serta sistem rekomendasi berbasis kesamaan preferensi pengguna [11]. Contoh segmentasi nasabah ditunjukkan pada Gambar 2.



Gambar 2: Segmentasi pelanggan memakai unsupervised learning untuk mengenali kelompok karakteristik nasabah [11].

## 2.2 Konsep Dasar Klusterisasi

Klusterisasi membagi sekumpulan objek observasi ke dalam sejumlah kelompok atau kluster. Analisis kluster berpegang pada dua kriteria geometris:

1. **Intra-cluster similarity tinggi:** Titik data di dalam kelompok yang sama saling berdekatan dan memiliki karakteristik fitur yang serupa.
2. **Inter-cluster dissimilarity tinggi:** Titik data yang berada pada kelompok berbeda terpisah jauh dengan perbedaan karakteristik yang tegas.

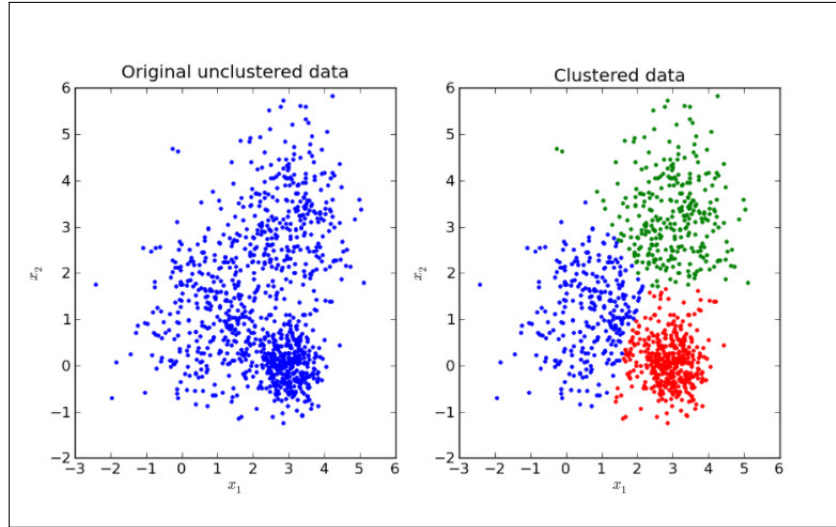
Metode klusterisasi dapat dikelompokkan ke dalam tiga kategori utama:

- **Partitional Clustering:** Membagi ruang data secara langsung ke dalam  $K$  kelompok tanpa struktur bertingkat, contohnya K-Means dan K-Medoids.
- **Density-Based Clustering:** Membentuk kelompok berdasarkan daerah dengan kepadatan titik data tinggi yang dipisahkan oleh daerah renggang, contohnya DBSCAN.
- **Hierarchical Clustering:** Membangun struktur pohon kluster (dendrogram) secara aglomeratif (penggabungan dari bawah) atau divisif (pembagian dari atas).

## 2.3 Algoritma K-Means

Algoritma K-Means mempartisi  $n$  data observasi ke dalam  $K$  kluster berdasarkan jarak terdekat terhadap titik pusat massa rata-rata (centroid) [3]. Konsep partisi ruang fitur

K-Means tampak pada Gambar 3.



Gambar 3: Konsep partisi ruang fitur pada algoritma K-Means [13].

Tahapan algoritma K-Means berjalan secara berulang:

1. **Inisialisasi Centroid:** Menentukan jumlah kluster  $K$ , kemudian memilih  $K$  titik awal secara acak di dalam ruang data sebagai koordinat centroid  $\mu_1, \mu_2, \dots, \mu_K$ .
2. **Penetapan Titik (Assignment Step):** Menghitung jarak Euclidean dari setiap titik  $x_i$  ke masing-masing centroid  $\mu_j$ :

$$d(x_i, \mu_j) = \sqrt{\sum_{f=1}^d (x_{i,f} - \mu_{j,f})^2} \quad (1)$$

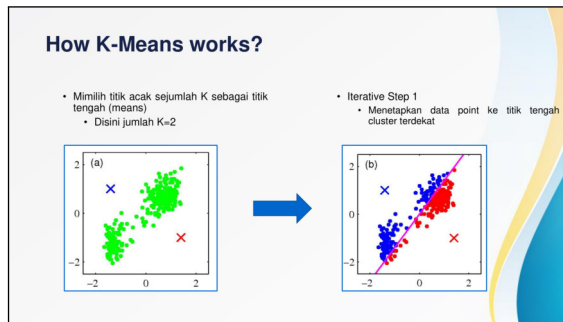
Setiap titik dialokasikan ke kluster dengan centroid terdekat, seperti diilustrasikan pada Gambar 4.

3. **Pembaruan Centroid (Update Step):** Menghitung ulang koordinat centroid  $\mu_j$  sebagai rata-rata aritmetika (mean vector) dari seluruh titik anggota kluster  $C_j$ :

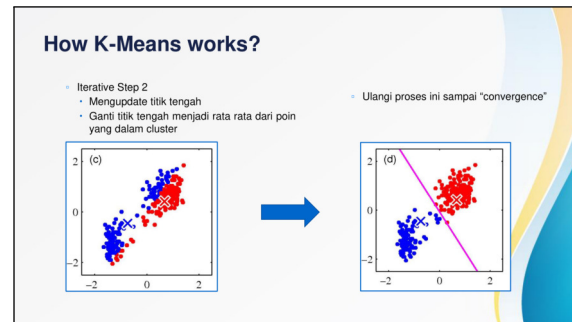
$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x \quad (2)$$

Proses pembaruan posisi centroid ini tampak pada Gambar 5.

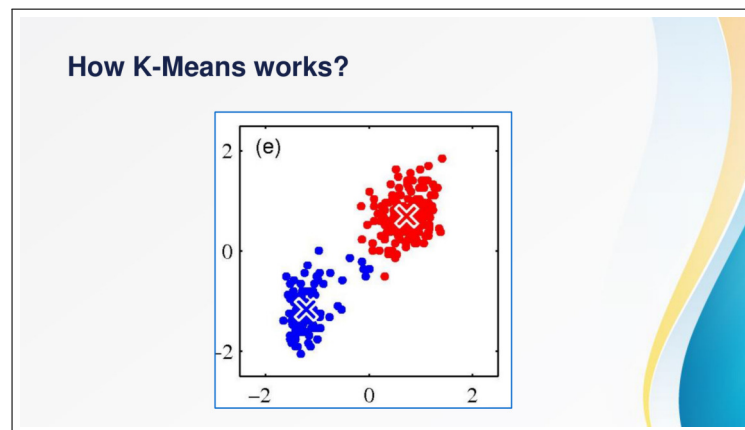
4. **Konvergensi:** Mengulang langkah kedua dan ketiga hingga posisi centroid tidak lagi berpindah secara signifikan, seperti terlihat pada Gambar 6.



Gambar 4: Penentuan centroid awal dan penetapan titik terdekat [12].



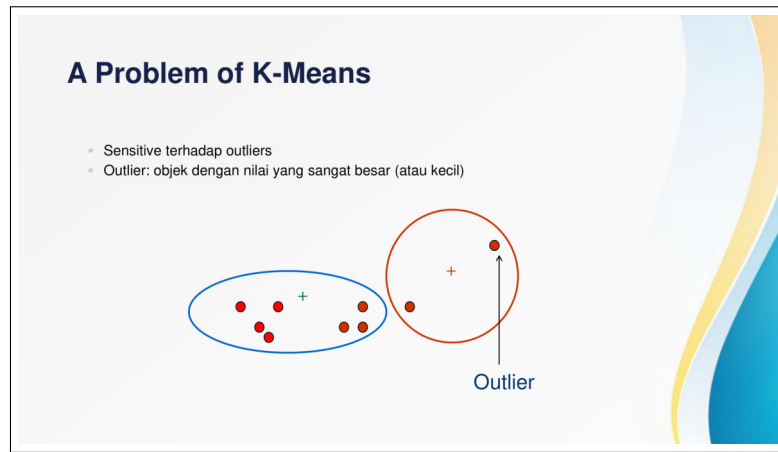
Gambar 5: Pembaruan centroid berdasarkan rata-rata anggota kluster [12].



Gambar 6: Kondisi konvergensi K-Means saat posisi partisi kluster telah stabil [12].

K-Means memiliki waktu komputasi yang efisien berorde  $O(t \cdot K \cdot n \cdot d)$ , dengan  $t$  jumlah iterasi,  $K$  jumlah kluster,  $n$  jumlah sampel, dan  $d$  dimensi fitur [11]. Namun K-Means memiliki kelemahan:

- Nilai  $K$  harus ditentukan sebelum algoritma dijalankan.
- Hasil pengelompokan dapat terjebak pada optimum lokal karena sensitif terhadap pemilihan lokasi centroid awal.
- K-Means mengasumsikan kluster berbentuk bola cembung (spherical) dengan ukuran yang seimbang, sehingga gagal pada data dengan pola bergelombang atau cincin.
- K-Means sensitif terhadap nilai pencilan (outlier). Karena centroid dihitung memakai rata-rata aritmetika, satu nilai ekstrem dapat menarik posisi centroid menjauh dari kumpulan data utama, seperti tampak pada Gambar 7.



Gambar 7: Pengaruh data penciran yang menggeser posisi centroid pada K-Means [12].

## 2.4 Evaluasi Kualitas Kluster dan Penentuan Nilai K

Evaluasi kluster pada data tanpa label dilakukan memakai kriteria evaluasi internal, yaitu Inertia dan Silhouette Score.

### 2.4.1 Inertia (Within-Cluster Sum of Squares)

Inertia mengukur tingkat kerapatan titik data di sekitar centroid klasternya. Nilai ini dihitung sebagai jumlah kuadrat jarak Euclidean antara setiap titik dengan centroid kluster terdekatnya:

$$\text{Inertia} = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \quad (3)$$

Nilai inertia selalu menurun ketika nilai  $K$  dinaikkan. Pada kondisi ekstrem saat  $K = n$ , inertia bernilai nol. Oleh karena itu, penurunan inertia dianalisis memakai grafik Elbow Method untuk mencari titik perubahan laju penurunan.

### 2.4.2 Silhouette Score

Silhouette Score mengukur tingkat pemisahan antar-kluster dibandingkan dengan kerapatan di dalam kluster sendiri [5]. Untuk sampel  $i$ , koefisien siluet dihitung melalui:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (4)$$

dengan  $a(i)$  menyatakan rata-rata jarak titik  $i$  ke semua titik lain di kluster yang sama, dan  $b(i)$  menyatakan rata-rata jarak titik  $i$  ke semua titik pada kluster tetangga terdekat.

Rentang nilai Silhouette Score berada pada interval  $[-1, 1]$ :

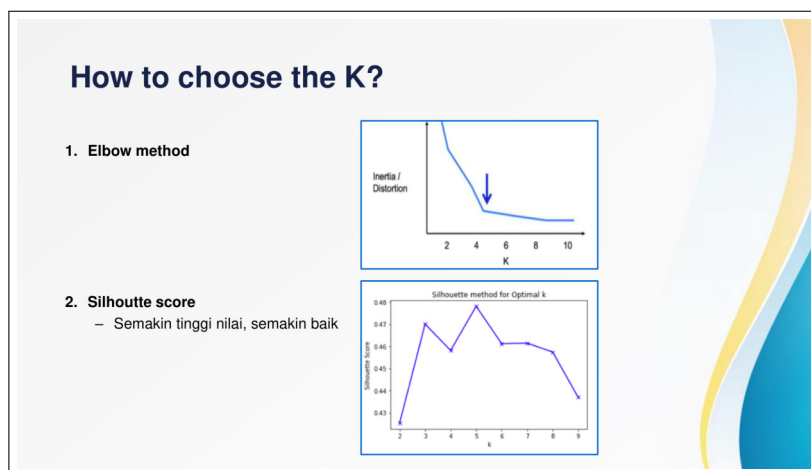
- Nilai mendekati 1 menunjukkan titik berada jauh dari kluster lain dan berdekatan dengan anggota kelompoknya.

- Nilai di sekitar 0 menandakan titik berada pada perbatasan antar-kluster.
- Nilai bernilai negatif menunjukkan titik kemungkinan dialokasikan pada kluster yang keliru karena jaraknya lebih dekat ke kluster lain.

### 2.4.3 Penentuan Nilai K Optimal

Dua metode utama dipakai untuk menentukan nilai  $K$  optimal:

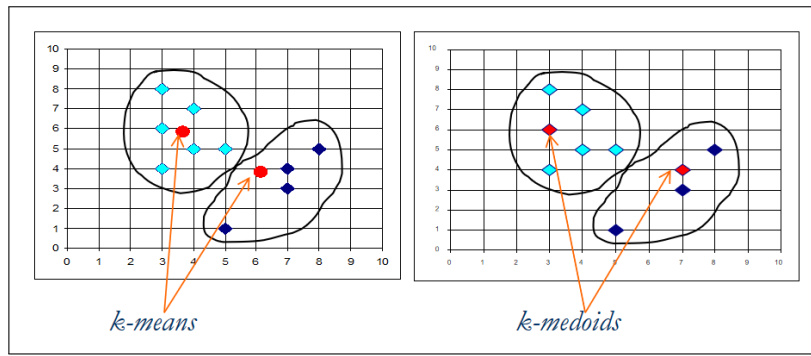
1. **Elbow Method:** Membuat plot grafik inertia terhadap variasi nilai  $K$ . Titik siku (elbow point) dipilih pada posisi di mana penurunan inertia mulai melambat secara nyata, seperti diilustrasikan pada Gambar 8.
2. **Deteksi Siku Otomatis (KneeLocator):** Algoritma ‘kneed’ menghitung kelengkungan maksimum kurva secara matematis untuk menemukan titik siku tanpa bias penglihatan manusia [10].
3. **Silhouette Analysis:** Memilih nilai  $K$  yang menghasilkan rata-rata skor siluet tertinggi di antara rentang uji.



Gambar 8: Penentuan jumlah kluster optimal memakai kurva Elbow dan Silhouette Score [11].

## 2.5 Algoritma K-Medoids (Partitioning Around Medoids)

Algoritma K-Medoids atau PAM dikembangkan oleh Leonard Kaufman dan Peter J. Rousseeuw pada tahun 1990 untuk mengatasi kelemahan K-Means terhadap pencilon [4]. K-Medoids memakai titik data riil dari dataset sebagai representasi pusat kluster (medoid), bukan rata-rata imajiner seperti pada K-Means. Perbedaan kedua titik pusat ini tampak pada Gambar 9.



Gambar 9: Perbedaan titik pusat: K-Means memakai rata-rata imajiner, sedangkan K-Medoids memakai data riil medoid [4].

Tabel 1 merangkum perbandingan teknis antara K-Means dan K-Medoids.

| Karakteristik      | K-Means                                           | K-Medoids (PAM)                                       |
|--------------------|---------------------------------------------------|-------------------------------------------------------|
| Pusat Kluster      | Vektor rata-rata (mean imajiner)                  | Objek data aktual di dalam dataset (medoid)           |
| Fungsi Objektif    | Meminimalkan jumlah kuadrat galat (Euclidean)     | Meminimalkan jumlah dissimilaritas absolut            |
| Ketahanan Pencilan | Peka terhadap pencilan [12]                       | Tahan terhadap pencilan dan derau [14]                |
| Kompleksitas       | Cepat, linier $O(t \cdot K \cdot n \cdot d)$ [15] | Lebih berat, kuadratik $O(t \cdot K \cdot (n - K)^2)$ |

Tabel 1: Perbandingan karakteristik algoritma K-Means dan K-Medoids.

Prinsip kerja PAM meliputi:

1. Memilih secara acak  $K$  objek data riil sebagai medoid awal  $M = \{m_1, \dots, m_K\}$ .
2. Menetapkan setiap objek non-medoid ke medoid terdekatnya.
3. Mengevaluasi pertukaran medoid (swapping): untuk setiap medoid  $m_i$  dan kandidat non-medoid  $o_h$ , posisi ditukar untuk sementara dan kriteria galat absolut total dihitung:

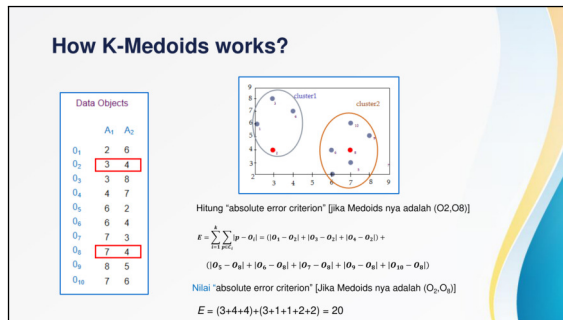
$$E = \sum_{i=1}^K \sum_{p \in C_i} |p - m_i| \quad (5)$$

4. Menghitung perubahan biaya pertukaran  $S = E_{\text{baru}} - E_{\text{lama}}$ . Jika  $S < 0$ , pertukaran menurunkan total galat sehingga posisi medoid diperbarui secara permanen. Jika  $S \geq 0$ , pertukaran ditolak.
5. Mengulang evaluasi hingga tidak ada lagi pertukaran yang menurunkan galat.

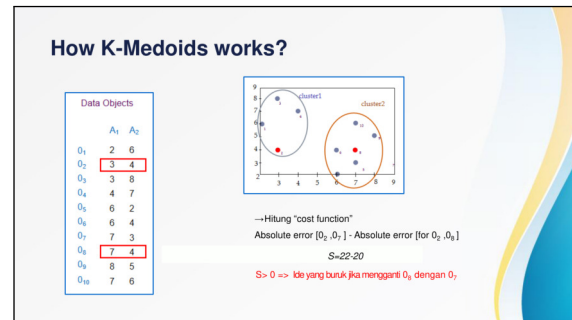
Sebagai simulasi numerik pada sepuluh titik data dua dimensi:  $O_1(2, 6)$ ,  $O_2(3, 4)$ ,  $O_3(3, 8)$ ,  $O_4(4, 7)$ ,  $O_5(6, 2)$ ,  $O_6(6, 4)$ ,  $O_7(7, 3)$ ,  $O_8(7, 4)$ ,  $O_9(8, 5)$ , dan  $O_{10}(7, 6)$ . Untuk  $K = 2$  dengan medoid awal ( $O_2, O_8$ ), total jarak pada kluster 1 bernilai 11 dan kluster 2 bernilai 9, sehingga  $E_{lama} = 20$ . Ketika medoid  $O_8$  dicoba ditukar dengan  $O_7(7, 3)$ , total jarak kluster 1 tetap 11 dan kluster 2 menjadi 11, sehingga  $E_{baru} = 22$ . Nilai biaya swapping menjadi:

$$S = E_{baru} - E_{lama} = 22 - 20 = +2 \quad (6)$$

Karena  $S > 0$ , pertukaran tersebut ditolak dan  $O_8$  tetap dipertahankan sebagai medoid. Proses evaluasi ini diilustrasikan pada Gambar 10 dan Gambar 11.



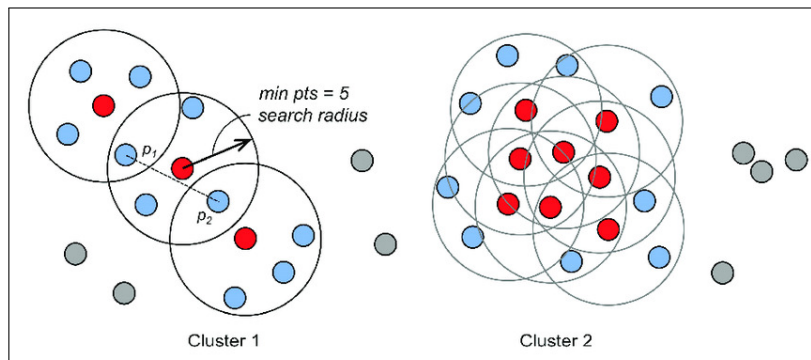
Gambar 10: Perhitungan galat absolut  $E$  pada ( $O_2, O_8$ ) [4].



Gambar 11: Evaluasi fungsi biaya pertukaran  $S > 0$  [4].

## 2.6 Algoritma DBSCAN

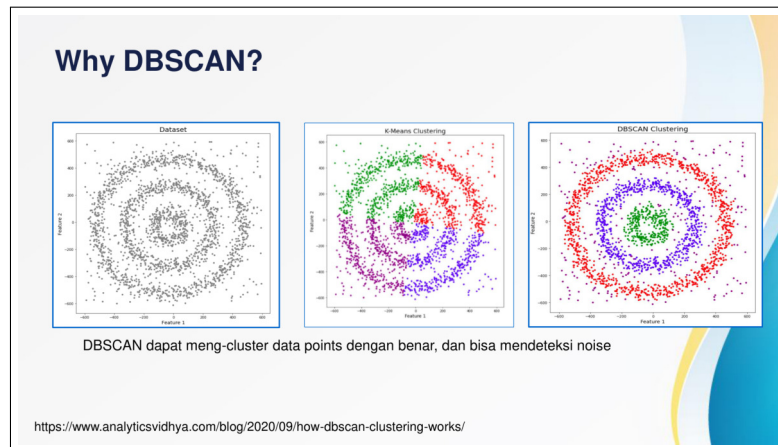
Algoritma DBSCAN diusulkan oleh Martin Ester, Hans-Peter Kriegel, Jörg Sander, dan Xiaowei Xu pada tahun 1996 [6]. Algoritma ini dirancang untuk mendeteksi kluster dengan bentuk arbitrer dan memisahkan data pencilan secara otomatis. Ilustrasi konsep klusterisasi berbasis kepadatan tampak pada Gambar 12.



Gambar 12: Konsep klusterisasi berbasis kepadatan spasial pada DBSCAN [6].

K-Means membagi data menjadi partisi sferis. Saat berhadapan dengan data yang membentuk cincin konsentris atau struktur bulan sabit, K-Means memotong kelompok menjadi irisan baji yang keliru. DBSCAN mengikuti rantai kepadatan titik, sehingga dapat

membedakan cincin dalam dan cincin luar dengan benar serta mengisolasi pencilan sebagai noise, seperti tampak pada Gambar 13.



Gambar 13: Perbandingan K-Means dan DBSCAN pada struktur data cincin konsentris [6].

DBSCAN memakai dua parameter utama:

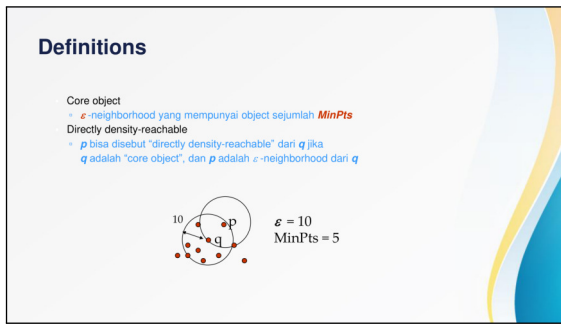
- $\epsilon$  (Epsilon): Radius jarak maksimum di sekitar suatu titik yang menjadi batas tetangga (neighborhood).
- $MinPts$  (Minimum Points): Jumlah minimum titik tetangga di dalam radius  $\epsilon$  agar sebuah titik memenuhi kualifikasi sebagai titik inti (core point).

Definisi lingkungan dan titik inti:

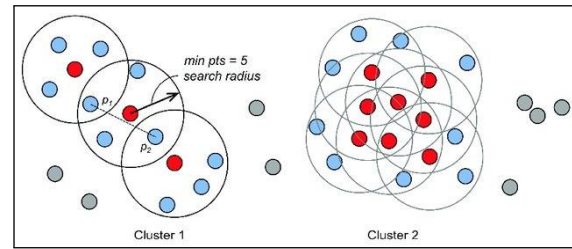
- $\epsilon$ -Neighborhood:  $N_\epsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}$ .
- Titik Inti (Core Point): Titik  $p$  yang memiliki setidaknya  $MinPts$  tetangga, yaitu  $|N_\epsilon(p)| \geq MinPts$ .
- Directly Density-Reachable: Titik  $p$  terjangkau langsung dari  $q$  jika  $q$  adalah core point dan  $p \in N_\epsilon(q)$ .
- Density-Reachable: Titik  $p$  terjangkau melalui rantai titik perantara yang saling directly density-reachable dari  $q$ .
- Density-Connected: Titik  $p$  dan  $q$  sama-sama density-reachable dari satu titik inti  $o$ .

Berdasarkan definisi tersebut, titik pada DBSCAN dikelompokkan ke dalam tiga kategori:

1. **Core Point:** Memiliki minimal  $MinPts$  tetangga di dalam radius  $\epsilon$ .
2. **Border Point:** Berada di dalam radius  $\epsilon$  dari titik inti, tetapi jumlah tetangganya kurang dari  $MinPts$ .
3. **Noise Point:** Bukan titik inti dan tidak terjangkau oleh titik inti mana pun.

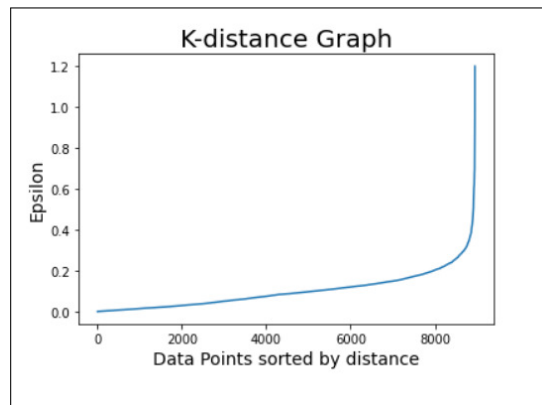


Gambar 14: Titik inti batas  $\epsilon$  dan  $MinPts$  [6].



Gambar 15: Tiga tipe titik DBSCAN [7].

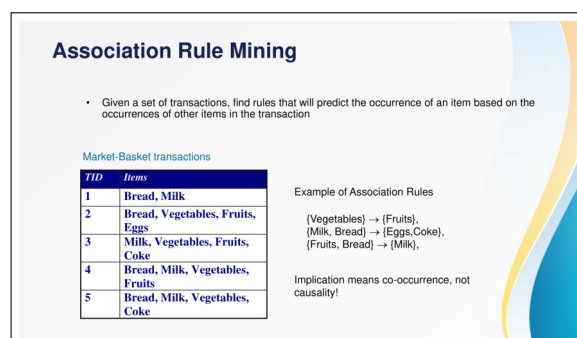
Penentuan parameter  $MinPts$  mengikuti aturan praktis: untuk data 2 dimensi digunakan  $MinPts = 4$  [6], sedangkan untuk dimensi  $D > 2$  disarankan  $MinPts = 2 \times D$  [7]. Nilai  $\epsilon$  ditentukan memakai grafik jarak k-tetangga terurut (sorted k-distance graph) dengan  $k = MinPts$ . Nilai  $\epsilon$  optimal diambil pada titik kelokan kurva, seperti tampak pada Gambar 16.



Gambar 16: Grafik sorted k-distance untuk menentukan nilai  $\epsilon$  optimal [6].

## 2.7 Association Rule Mining

Association Rule Mining menemukan keterkaitan kemunculan bersama antar-item dalam basis data transaksi transaksi keranjang belanja (Market Basket Analysis) [8]. Contoh data transaksi belanja ditunjukkan pada Gambar 17.



Gambar 17: Contoh basis data transaksi keranjang belanja [12].

Aturan asosiasi ditulis dalam bentuk implikasi logis:

$$X \rightarrow Y, \quad \text{dengan } X \cap Y = \emptyset \quad (7)$$

dengan  $X$  sebagai antecedent dan  $Y$  sebagai consequent. Pernyataan ini menyatakan kecenderungan kemunculan bersama (co-occurrence), bukan hubungan sebab-akibat kausalitas.

Tiga metrik kualitas aturan asosiasi meliputi:

1. **Support (s):** Fraksi transaksi dalam database yang memuat kombinasi  $X$  dan  $Y$  bersamaan:  $s(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{|T|}$ .
2. **Confidence (c):** Peluang bersyarat kemunculan  $Y$  pada transaksi yang memuat  $X$ :  $c(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}$ .
3. **Lift:** Kekuatan aturan terhadap asumsi saling bebas:  $Lift(X \rightarrow Y) = \frac{c(X \rightarrow Y)}{s(Y)} = \frac{s(X \cup Y)}{s(X) \cdot s(Y)}$ . Nilai  $Lift > 1$  menandakan asosiasi positif,  $Lift = 1$  independen, dan  $Lift < 1$  substitutif.

Pada basis data berisi lima transaksi: TID 1: {Bread, Milk}, TID 2: {Bread, Vegetables, Fruits, Eggs}, TID 3: {Milk, Vegetables, Fruits, Coke}, TID 4: {Bread, Milk, Vegetables, Fruits}, dan TID 5: {Bread, Milk, Vegetables, Coke}. Perhitungan untuk aturan {Milk, Vegetables}  $\rightarrow$  {Fruits} menghasilkan Support =  $2/5 = 0,40$ , Confidence =  $2/3 \approx 0,67$ , dan  $Lift = 0,40/(0,60 \times 0,60) \approx 1,11$ . Nilai  $Lift > 1$  mengonfirmasi bahwa aturan ini memiliki asosiasi positif yang valid. Rincian metrik dan contoh perhitungannya ditampilkan pada Gambar 18 dan Gambar 19.

| Definition: Association Rule   |                                                                                                      |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Rule Evaluation Metrics</b> |                                                                                                      |
| – Support (s)                  | $support(A \rightarrow C) = support(A \cup C), \text{ range: } [0,1]$                                |
| – Confidence (c)               | $confidence(A \rightarrow C) = \frac{support(A \rightarrow C)}{support(A)}, \text{ range: } [0,1]$   |
| – Lift                         | $lift(A \rightarrow C) = \frac{confidence(A \rightarrow C)}{support(C)}, \text{ range: } [0,\infty]$ |

Gambar 18: Metrik evaluasi aturan asosiasi [12].

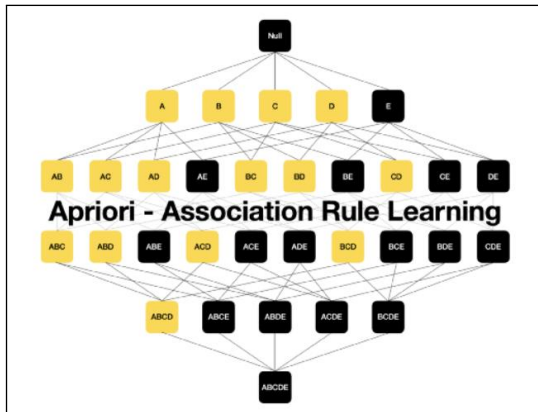
| Definition: Association Rule                                                                                      |  |
|-------------------------------------------------------------------------------------------------------------------|--|
| <b>Example:</b>                                                                                                   |  |
| $\{Milk, Vegetables\} \Rightarrow Fruits$                                                                         |  |
| $\{X\} \Rightarrow Y$                                                                                             |  |
| $s = \frac{\sigma(Milk, Vegetables, Fruits)}{ T } = \frac{2}{5} = 0.4$                                            |  |
| $c = \frac{\sigma(Milk, Vegetables, Fruits)}{\sigma(Milk, Vegetables)} = \frac{2}{3} = 0.67$                      |  |
| $l = \frac{\sigma(Milk, Vegetables, Fruits)}{\sigma(Milk, Vegetables) * \sigma(Fruits)} = \frac{2}{(3*3)} = 0.22$ |  |

Gambar 19: Langkah perhitungan metrik [12].

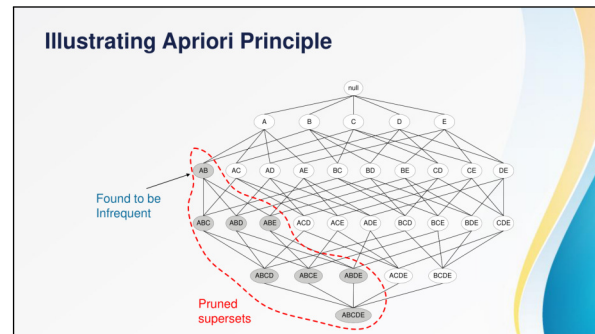
## 2.8 Algoritma Apriori

Untuk basis data dengan  $d$  item berbeda, ruang pencarian memuat  $2^d - 1$  kemungkinan kombinasi itemset. Algoritma Apriori diperkenalkan oleh Rakesh Agrawal dan Ramakrishnan Srikant pada tahun 1994 untuk menekan ledakan kombinatorial ini [9]. Prinsip Apriori berpegang pada sifat anti-monoton nilai support: jika suatu itemset bersifat frequent, maka seluruh himpunan bagiannya (subset) pasti frequent. Sebaliknya, jika suatu

itemset tidak frequent, maka seluruh superset yang memuatnya dipastikan tidak frequent dan langsung dipangkas (lattice pruning), seperti diilustrasikan pada Gambar 20 dan Gambar 21.

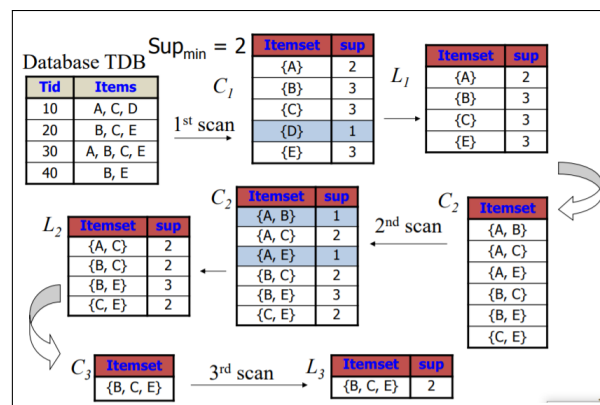


Gambar 20: Prinsip Apriori pada support [9].



Gambar 21: Pemangkas ruang superset [12].

Alur kerja iteratif algoritma Apriori menghasilkan frequent 1-itemset ( $L_1$ ), lalu membangkitkan kandidat  $C_{k+1}$  melalui langkah penggabungan (join), memangkas kandidat yang memuat subset tidak frequent (prune), dan memindai database untuk menyaring  $L_{k+1}$ , seperti tampak pada Gambar 22.



Gambar 22: Alur kerja algoritma Apriori: pembangkitan kandidat dan penyaringan bertahap [9].

# Alat dan Bahan

Sesuai Modul Pertemuan 6 Praktikum Penambangan Data, alat dan bahan yang digunakan dalam praktikum ini meliputi:

## A. Perangkat Keras

1. Komputer jinjing (laptop) dengan kapasitas memori minimal 4 GB RAM (direkomendasikan 8 GB atau lebih).
2. Koneksi internet yang stabil untuk mengakses lingkungan komputasi daring dan mengunduh berkas dataset.

## B. Perangkat Lunak

1. Sistem operasi modern (Windows, macOS, atau Linux).
2. Peramban web (Google Chrome, Mozilla Firefox, atau Microsoft Edge).
3. Lingkungan pengembangan interaktif Python: Google Colaboratory dan Jupyter Notebook lokal.
4. Bahasa pemrograman Python versi 3.8 ke atas [23].
5. Pustaka pemrograman Python:
  - pandas dan numpy untuk manipulasi data tabular dan operasi matriks numerik [24, 25].
  - matplotlib.pyplot dan seaborn untuk pembuatan grafik 2 dimensi [26, 27].
  - plotly.express untuk pembuatan visualisasi sebaran kluster interaktif 3 dimensi [28].
  - scikit-learn untuk standarisasi skala (StandardScaler), pengkodean label (LabelEncoder), pemodelan KMeans, dan metrik silhouette\_score [16, 17, 18].
  - scikit-learn-extra untuk implementasi algoritma KMedoids [19].
  - kneed (KneeLocator) untuk mendeteksi koordinat titik siku kurva secara otomatis [20].

# Hasil dan Pembahasan

## 4.1 Percobaan Latihan Praktikum

Percobaan latihan praktikum mengimplementasikan segmentasi nasabah kartu kredit memakai *Credit Card Dataset for Clustering* (CC\_GENERAL.csv) [21]. Seluruh tahapan eksperimen latihan dan tugas pada laporan ini telah diimplementasikan serta dapat diakses dan dijalankan secara interaktif melalui Google Colaboratory pada tautan berikut:

[https://colab.research.google.com/drive/143E58VDNkkjOdFiezQN4yGggLJ6\\_xo5j?usp=sharing](https://colab.research.google.com/drive/143E58VDNkkjOdFiezQN4yGggLJ6_xo5j?usp=sharing) [29]

### 4.1.1 Persiapan dan Impor Pustaka

Tahap awal menginstal pustaka tambahan ‘scikit-learn-extra’ dan ‘kneed’, lalu mengimpor pustaka utama untuk manipulasi data, standarisasi, pemodelan klaster, dan visualisasi.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import plotly.express as px
6
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.cluster import KMeans
9 from sklearn.metrics import silhouette_samples, silhouette_score
```

Listing .1: Impor pustaka praktikum klasterisasi.

```

1.1 Instalasi Library Tambahan
Instalasi pustaka scikit-learn-extra (untuk K-Medoids) dan kneed (untuk deteksi titik siku otomatis).

!!!
~$ pip install scikit-learn-extra kneed

Collecting scikit-learn-extra
  Downloading scikit-learn-extra-0.3.0.tar.gz (818 KB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting kneed
  Downloading kneed-0.6.0-py3-none-any.whl.metadata (6.4 KB)
Requirement already satisfied: numpy<=1.3.3 in /usr/local/lib/python3.11/dist-packages (from scikit-learn-extra) (2.1.1)
Requirement already satisfied: scipy<=0.19.1 in /usr/local/lib/python3.11/dist-packages (from scikit-learn-extra) (1.10.3)
Requirement already satisfied: scikit-learn<=0.23.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn-extra) (1.6.1)
Requirement already satisfied: joblib<=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn-extra) (1.6.0)
Requirement already satisfied: threadpoolctl<=2.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn-extra) (3.6.0)
Requirement already satisfied: cloudpickle<=3.0 in /usr/local/lib/python3.11/dist-packages (from kneed) (3.1.2)
Downloading kneed-0.6.0-py3-none-any.whl (28 KB)
Building wheels for collected packages: scikit-learn-extra
  Building wheel for scikit-learn-extra (pyproject.toml) ... done
  Created wheel for scikit-learn-extra: filename=scikit_learn_extra-0.3.0-cp311-cp311-linux_x86_64.whl size=2154566 sha256=863db886762ddcc198882e21a1959966a2b587ea2e71c15c9f628ed3
  Stored in directory: /root/.cache/pip/wheels/17/4b/b8/6d71581d9981b18c9c91ad6d1eb88a71547e1da381fc
Successfully built scikit-learn-extra
Installing collected packages: kneed, scikit-learn-extra
Successfully installed kneed-0.6.0 scikit-learn-extra-0.3.0

1.2 Import Library
Mengimpor modul-modul yang dibutuhkan sepanjang praktikum.

!!!
~$
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples, silhouette_score
from sklearn_extra.cluster import KMedoids
from kneed import KneeLocator

plt.style.use('ggplot')
matplotlib inline

```

Gambar 23: Tangkapan layar impor pustaka utama pada Google Colab.

## 4.1.2 Membaca dan Memuat Informasi Dataset

Dataset dibaca langsung dari tautan repositori daring menggunakan pandas. Dataset memuat 8.950 baris observasi dan 18 kolom fitur.

```

1 url = ('https://raw.githubusercontent.com/ganjar87/'
2       'data_science_practice/main/CC%20GENERAL.csv')
3 df = pd.read_csv(url)
4 print("Dimensi dataset:", df.shape)
5 df.head()

```

Listing .2: Membaca dataset kartu kredit.

```

1.3 Membaca dan Memuat Dataset
Dataset dibaca dari repositori mirror data science practice CC GENERAL.csv.

!!!
~$ url = "https://raw.githubusercontent.com/ganjar87/data_science_practice/main/CC%20GENERAL.csv"
df = pd.read_csv(url)
print("Dimensi dataset:", df.shape)
df.head()

Dimensi dataset: (8950, 18)

```

|   | CUST_ID | BALANCE     | BALANCE_FREQUENCY | PURCHASES | ONEOFF_PURCHASES | INSTALLMENTS_PURCHASES | CASH_ADVANCE | PURCHASES_FREQUENCY | ONEOFF_PURCHASES_FREQUENCY | PURCHASES_INSTALLMENTS_FREQUENCY | CASH_ADVANCE_FREQUENCY | CASH_ADVANCE_TOT | PURCHASES_TOT |
|---|---------|-------------|-------------------|-----------|------------------|------------------------|--------------|---------------------|----------------------------|----------------------------------|------------------------|------------------|---------------|
| 0 | C10001  | 42000.00    | 0.818182          | 95.45     | 0.00             | 0.00                   | 0.000000     | 0.100000            | 0.000000                   | 0.000000                         | 0.000000               | 0.000000         | 0             |
| 1 | C10002  | 3000.467416 | 0.800000          | 0.00      | 0.00             | 0.00                   | 0.000000     | 0.000000            | 0.000000                   | 0.000000                         | 0.000000               | 0.000000         | 4             |
| 2 | C10003  | 2498.148802 | 1.000000          | 773.17    | 773.17           | 0.00                   | 0.000000     | 1.000000            | 1.000000                   | 0.000000                         | 0.000000               | 0.000000         | 0             |
| 3 | C10004  | 1696.670542 | 0.830564          | 1499.00   | 1499.00          | 0.00                   | 205.788017   | 0.083333            | 0.083333                   | 0.000000                         | 0.000000               | 0.083333         | 1             |
| 4 | C10005  | 817.714335  | 1.000000          | 16.00     | 16.00            | 0.00                   | 0.000000     | 0.083333            | 0.083333                   | 0.000000                         | 0.000000               | 0.000000         | 0             |

Gambar 24: Tangkapan layar pembacaan data dan lima baris pertama CC GENERAL.csv.

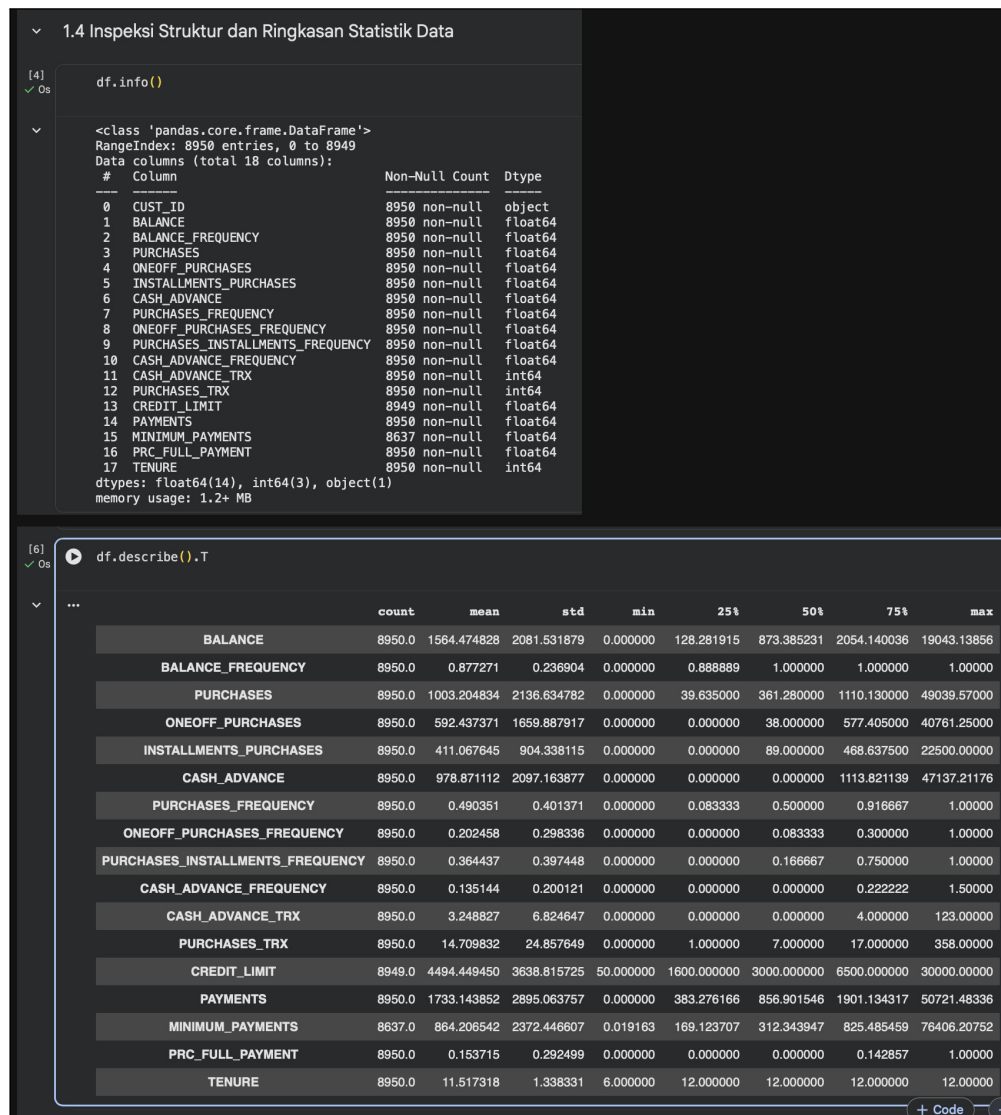
Saya memeriksa struktur tipe data dan sebaran ringkasan statistik memakai `df.info()` dan `df.describe()`. Dari pemeriksaan ini, 14 fitur bertipe `float64`, 3 fitur bertipe `int64`, dan 1 fitur teks yaitu `CUST_ID`.

```

1 df.info()
2 df.describe().T

```

Listing .3: Pemeriksaan tipe data dan statistik deskriptif.

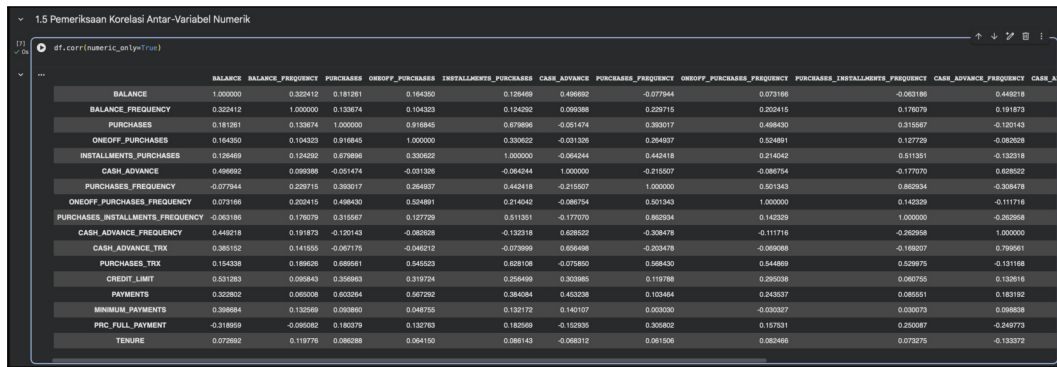


Gambar 25: Tangkapan layar struktur tipe data dan ringkasan statistik deskriptif.

Korelasi Pearson dihitung untuk mengenali hubungan antar-variabel numerik. Fitur PURCHASES memiliki korelasi linear tinggi dengan ONEOFF\_PURCHASES ( $r = 0,917$ ) dan berkorelasi moderat dengan PAYMENTS ( $r = 0,603$ ).

```
df.corr(numeric_only=True)
```

Listing .4: Pemeriksaan matriks korelasi antar-fitur.



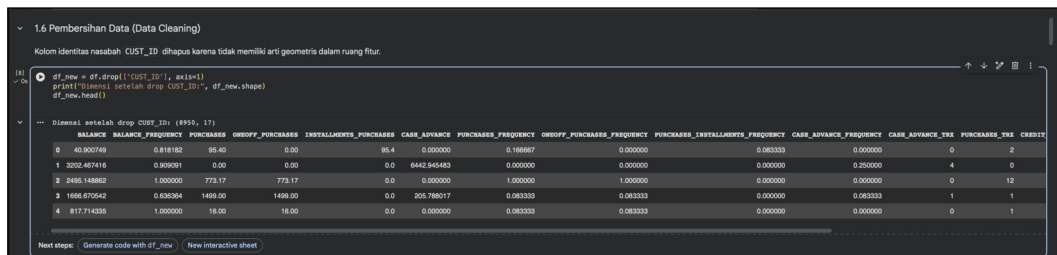
Gambar 26: Tangkapan layar matriks korelasi antar-variabel numerik.

### 4.1.3 Pembersihan Data (Data Cleaning)

Kolom CUST\_ID dihapus karena hanya berupa kode pengenalan nasabah yang tidak memiliki makna jarak geometris dalam klusterisasi.

```
1 df_new = df.drop('CUST_ID', axis=1)
2 df_new.head()
```

Listing .5: Menghapus kolom CUST\_ID.



Gambar 27: Tangkapan layar penghapusan kolom CUST\_ID.

Pemeriksaan nilai hilang mendeteksi 1 nilai kosong pada kolom CREDIT\_LIMIT dan 313 nilai kosong pada kolom MINIMUM\_PAYMENTS.

```
1 df_new.isnull().sum()
```

Listing .6: Pemeriksaan jumlah nilai hilang pada setiap kolom.

1.7 Identifikasi Nilai Hilang (Missing Values)

[9] df\_new.isnull().sum()

|                                  |     |
|----------------------------------|-----|
| ...                              | 0   |
| BALANCE                          | 0   |
| BALANCE_FREQUENCY                | 0   |
| PURCHASES                        | 0   |
| ONEOFF_PURCHASES                 | 0   |
| INSTALLMENTS_PURCHASES           | 0   |
| CASH_ADVANCE                     | 0   |
| PURCHASES_FREQUENCY              | 0   |
| ONEOFF_PURCHASES_FREQUENCY       | 0   |
| PURCHASES_INSTALLMENTS_FREQUENCY | 0   |
| CASH_ADVANCE_FREQUENCY           | 0   |
| CASH_ADVANCE_TRX                 | 0   |
| PURCHASES_TRX                    | 0   |
| CREDIT_LIMIT                     | 1   |
| PAYMENTS                         | 0   |
| MINIMUM_PAYMENTS                 | 313 |
| PRC_FULL_PAYMENT                 | 0   |
| TENURE                           | 0   |

dtype: int64

Gambar 28: Tangkapan layar pemeriksaan nilai hilang sebelum imputasi.

Imputasi nilai hilang dilakukan memakai nilai median dari masing-masing fitur bersangkutan. Median dipilih agar pengisian data tidak terpengaruh oleh sebaran nilai ekstrem (outlier). Setelah imputasi dijalankan, pemeriksaan ulang mengonfirmasi bahwa seluruh kolom telah terisi penuh.

```

1 df_new['MINIMUM_PAYMENTS'].fillna(df_new['MINIMUM_PAYMENTS'].median(), inplace=True)
2 df_new['CREDIT_LIMIT'].fillna(df_new['CREDIT_LIMIT'].median(), inplace=True)
3 df_new.isnull().sum()

```

Listing .7: Imputasi nilai hilang dengan median dan verifikasi hasil.

1.8 Imputasi Nilai Hilang dengan Nilai Median

Fitur MINIMUM\_PAYMENTS dan CREDIT\_LIMIT memiliki nilai kosong. Imputasi menggunakan median dipilih agar tidak terpengaruh oleh pencilan.

```

[10] df_new['MINIMUM_PAYMENTS'] = df_new['MINIMUM_PAYMENTS'].fillna(df_new['MINIMUM_PAYMENTS'].median())
      df_new['CREDIT_LIMIT'] = df_new['CREDIT_LIMIT'].fillna(df_new['CREDIT_LIMIT'].median())

      print("Pengecekan ulang missing value:")
      print(df_new.isnull().sum())

```

... Pengecekan ulang missing value:

Gambar 29: Tangkapan layar kode imputasi median.

|                                  |   |
|----------------------------------|---|
| BALANCE                          | 0 |
| BALANCE_FREQUENCY                | 0 |
| PURCHASES                        | 0 |
| ONEOFF_PURCHASES                 | 0 |
| INSTALLMENTS_PURCHASES           | 0 |
| CASH_ADVANCE                     | 0 |
| PURCHASES_FREQUENCY              | 0 |
| ONEOFF_PURCHASES_FREQUENCY       | 0 |
| PURCHASES_INSTALLMENTS_FREQUENCY | 0 |
| CASH_ADVANCE_FREQUENCY           | 0 |
| CASH_ADVANCE_TRX                 | 0 |
| PURCHASES_TRX                    | 0 |
| CREDIT_LIMIT                     | 0 |
| PAYMENTS                         | 0 |
| MINIMUM_PAYMENTS                 | 0 |
| PRC_FULL_PAYMENT                 | 0 |
| TENURE                           | 0 |
| dtype: int64                     |   |

Gambar 30: Tangkapan layar verifikasi data setelah imputasi selesai.

#### 4.1.4 Standarisasi Skala Fitur Data

Rentang nilai antar-fitur sangat timpang. Misalnya kolom BALANCE mencapai ribuan dolar, sedangkan frekuensi transaksi berada pada skala 0 sampai 1. Seluruh 17 fitur numerik distandarisasi memakai StandardScaler agar memiliki rata-rata  $\mu = 0$  dan simpangan baku  $\sigma = 1$ . Langkah ini mencegah fitur dengan satuan besar mendominasi perhitungan jarak Euclidean.

```

1 X = df_new.astype(float).values
2 scaler = StandardScaler().fit(X)
3 X_new = scaler.transform(X)
4 X_new

```

Listing .8: Standarisasi fitur memakai StandardScaler.

|                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.9 Standarisasi Skala Fitur (StandardScaler)                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Algoritma klusterisasi sensitif terhadap rentang skala. Seluruh fitur distandarisasi agar berdistribusi rata-rata nol dan standar deviasi satu. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| [11]<br>✓ Os                                                                                                                                    | <pre> X = df_new.astype(float).values scaler = StandardScaler().fit(X) X_new = scaler.transform(X) print("Bentuk matriks ternormalisasi:", X_new.shape) X_new </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| ✓                                                                                                                                               | <pre> Bentuk matriks ternormalisasi: (8950, 17) array([[ -0.73198937, -0.24943448, -0.42489974, ..., -0.3024    ,         -0.52555097,  0.36067954],        [ 0.78696085,  0.13432467, -0.46955188, ...,  0.09749953,         0.2342269 ,  0.36067954],        [ 0.44713513,  0.51808382, -0.10766823, ..., -0.0932934 ,         -0.52555097,  0.36067954],        ...,        [-0.7403981 , -0.18547673, -0.40196519, ..., -0.32687479,         0.32919999, -4.12276757],        [-0.74517423, -0.18547673, -0.46955188, ..., -0.33830497,         0.32919999, -4.12276757],        [-0.57257511, -0.88903307,  0.04214581, ..., -0.3243581 ,         -0.52555097, -4.12276757]]) </pre> |

Gambar 31: Tangkapan layar matriks fitur ternormalisasi StandardScaler.

#### 4.1.5 Penentuan Nilai K Optimal pada K-Means

Saya menguji model K-Means untuk rentang nilai  $K = 1$  sampai 10 dengan mencatat nilai inertia (WCSS) pada setiap percobaan.

```

1 inertia_list = []
2 for num_clusters in range(1, 11):
3     kmeans_model = KMeans(n_clusters=num_clusters, random_state=42)
4     kmeans_model.fit(X_new)
5     inertia_list.append(kmeans_model.inertia_)
6     print("For n_clusters = {}, inertia value is {}".format(
7         num_clusters, kmeans_model.inertia_))

```

Listing .9: Pengujian nilai inertia K-Means untuk K = 1 hingga 10.

```

1.10 Penentuan Nilai K Optimal: Elbow Method pada K-Means

Menguji nilai K = 1 sampai 10 dan mencatat nilai inertia (Within-Cluster Sum of Squares / WCSS).

[12] ✓ 1s inertia_list = []
      for num_clusters in range(1, 11):
          kmeans_model = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
          kmeans_model.fit(X_new)
          inertia_list.append(kmeans_model.inertia_)
          print(f"For n_clusters = {num_clusters}, inertia value is {kmeans_model.inertia_:.2f}")

... For n_clusters = 1, inertia value is 152150.00
For n_clusters = 2, inertia value is 127784.53
For n_clusters = 3, inertia value is 111975.04
For n_clusters = 4, inertia value is 99061.94
For n_clusters = 5, inertia value is 91490.50
For n_clusters = 6, inertia value is 84826.59
For n_clusters = 7, inertia value is 79856.16
For n_clusters = 8, inertia value is 74484.88
For n_clusters = 9, inertia value is 69828.70
For n_clusters = 10, inertia value is 66466.41

```

Gambar 32: Tangkapan layar pencatatan nilai inertia K-Means.

Hasil perhitungan inertia dirangkum pada Tabel 2.

| Jumlah Kluster (K) | Inertia (WCSS) | Silhouette Score |
|--------------------|----------------|------------------|
| 1                  | 152.150,00     | -                |
| 2                  | 127.784,71     | 0,2095           |
| 3                  | 111.975,04     | 0,2504           |
| 4                  | 99.061,94      | 0,1977           |
| 5                  | 91.491,11      | 0,1933           |
| 6                  | 84.825,07      | 0,2027           |
| 7                  | 79.551,38      | 0,2150           |
| 8                  | 74.467,92      | 0,2184           |
| 9                  | 69.832,80      | 0,2150           |
| 10                 | 66.459,92      | 0,2193           |

Tabel 2: Evaluasi nilai Inertia dan Silhouette Score pada K-Means.

Saya memplot kurva inertia untuk mengamati titik siku. Pendeteksian siku secara objektif dilakukan menggunakan modul KneeLocator dari pustaka kneed.

```

1 plt.plot(range(1, 11), inertia_list, marker='o', linewidth=2, markersize=8)
2 plt.xlabel("Number of Clusters", size=13)
3 plt.ylabel("Inertia Value", size=13)
4 plt.title("Inertia values vary depending on clusters utilized.")
5 plt.show()
6
7 from kneed import KneeLocator
8 kneedle = KneeLocator(
9     range(1, 11), inertia_list,
10    S=1.0, curve='convex', direction='decreasing'
11 )
12 print("Optimal Knee Point:", round(kneedle.knee, 3))
13 print("Optimal Elbow Point:", round(kneedle.elbow, 3))
14
15 plt.style.use('ggplot')
16 kneedle.plot_knee()
17 plt.show()

```

Listing .10: Pembuatan plot kurva inertia dan deteksi KneeLocator.

```

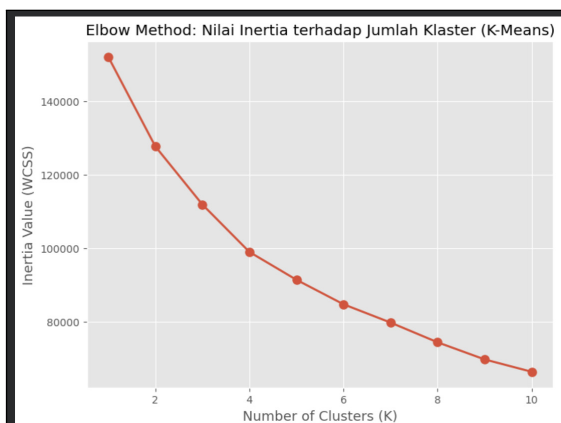
kneedle = KneeLocator(range(1, 11), inertia_list, S=1.0, curve='convex', direction='decreasing')
print("Optimal Knee Point:", round(kneedle.knee, 3))
print("Optimal Elbow Point:", round(kneedle.elbow, 3))

kneedle.plot_knee()
plt.title("Validasi Titik Siku Kurva K-Means (KneeLocator)")
plt.show()

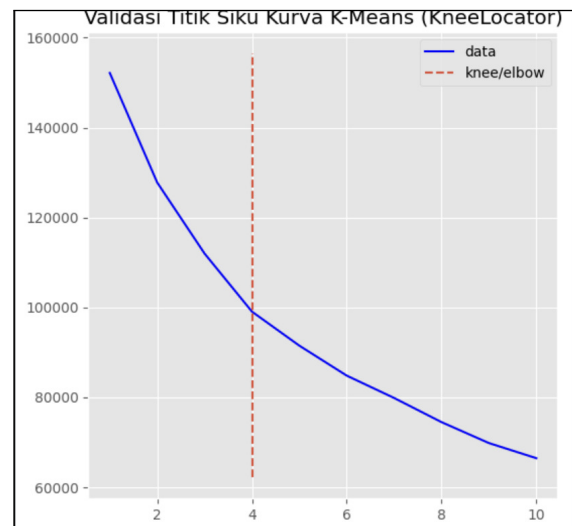
```

Optimal Knee Point: 4  
Optimal Elbow Point: 4

Gambar 33: Tangkapan layar luaran deteksi titik siku optimal KneeLocator K = 4.



Gambar 34: Kurva Elbow Method K-Means.



Gambar 35: Plot konfirmasi titik siku K = 4.

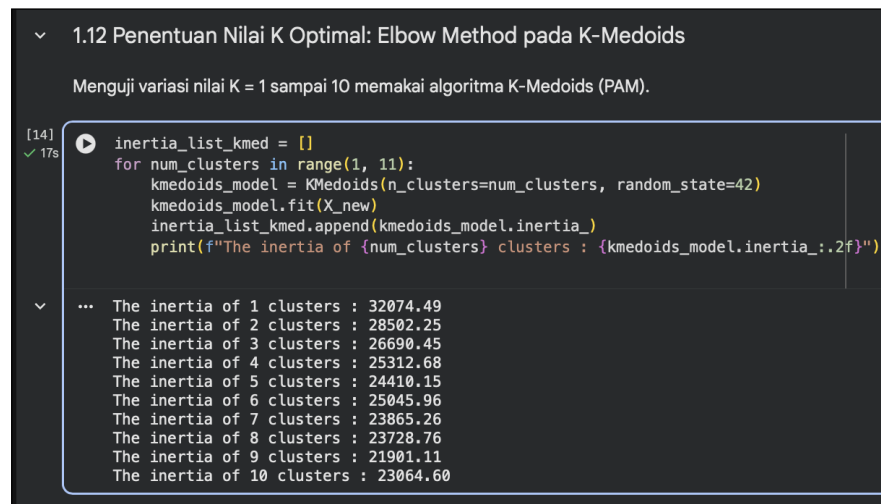
KneeLocator mengonfirmasi titik siku optimal pada  $K = 4$ . Setelah titik  $K = 4$ , laju penurunan inertia melandai.

### 4.1.6 Penentuan Nilai K Optimal pada K-Medoids

Eksperimen serupa dilakukan memakai KMedoids dari `sklearn_extra.cluster` untuk nilai  $K = 1$  sampai 10.

```
1 from sklearn_extra.cluster import KMedoids
2
3 inertia_list_kmed = []
4 for num_clusters in range(1, 11):
5     kmedoids_model = KMedoids(n_clusters=num_clusters, random_state=42)
6     kmedoids_model.fit(X_new)
7     inertia_list_kmed.append(kmedoids_model.inertia_)
8     print(f"The inertia of {num_clusters} clusters : {kmedoids_model.inertia_}")
```

Listing .11: Evaluasi nilai inertia K-Medoids.

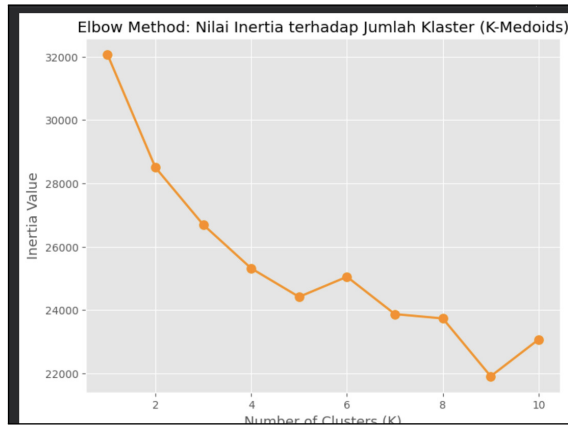


Gambar 36: Tangkapan layar perhitungan inertia K-Medoids.

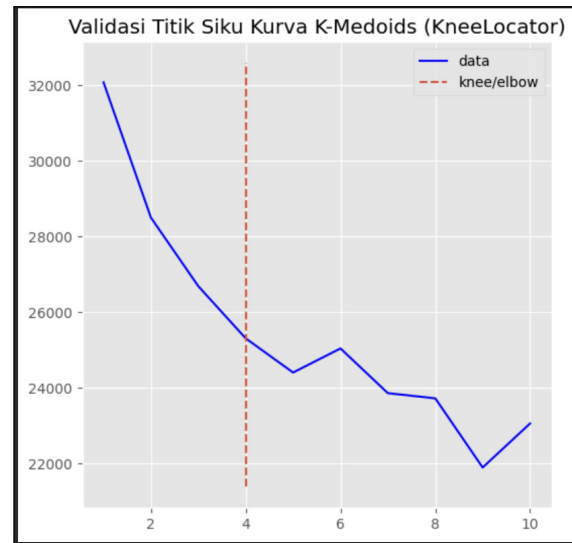
Nilai inertia K-Medoids pada  $K = 1$  bernilai 32.074,49, lalu turun ke 28.502,25 ( $K = 2$ ), 26.690,45 ( $K = 3$ ), 25.312,68 ( $K = 4$ ), dan 24.410,15 ( $K = 5$ ). Penerapan KneeLocator pada K-Medoids juga menghasilkan titik siku pada  $K = 4$ .

```
1 plt.plot(range(1, 11), inertia_list_kmed, marker='o', linewidth=2, markersize=8)
2 plt.xlabel("Number of Clusters", size=13)
3 plt.ylabel("Inertia Value", size=13)
4 plt.title("Inertia values vary depending on clusters utilized.")
5 plt.show()
6
7 kneedle_med = KneeLocator(
8     range(1, 11), inertia_list_kmed,
9     S=1.0, curve='convex', direction='decreasing'
10 )
11 print("Knee point K-Medoids:", kneedle_med.knee)
12 kneedle_med.plot_knee()
13 plt.show()
```

Listing .12: Visualisasi kurva elbow dan deteksi titik siku K-Medoids.



Gambar 37: Kurva Elbow Method K-Medoids.



Gambar 38: Validasi titik siku K-Medoids K = 4.

#### 4.1.7 Evaluasi Silhouette Score pada K-Means dan K-Medoids

Evaluasi skor siluet dilakukan pada rentang  $K = 2$  sampai 10.

```
1 sh_list = []
2 for num_clusters in range(2, 11):
3     kmeans = KMeans(n_clusters=num_clusters, random_state=42)
4     cluster_labels = kmeans.fit_predict(X_new)
5     score = silhouette_score(X_new, cluster_labels)
6     sh_list.append(score)
7     print("For n_clusters = {}, silhouette score is {}".format(
8         num_clusters, score))
9
10 plt.plot(range(2, 11), sh_list, marker='o', linewidth=2, markersize=8)
11 plt.xlabel("Number of Clusters", size=13)
12 plt.ylabel("silhouette score", size=13)
13 plt.title("Silhouette score vary depending on clusters utilized.")
14 plt.show()
```

Listing .13: Perhitungan dan visualisasi Silhouette Score K-Means.

```

1.14 Evaluasi Silhouette Score pada K-Means
Menghitung koefisien siluet untuk rentang K = 2 sampai 10.

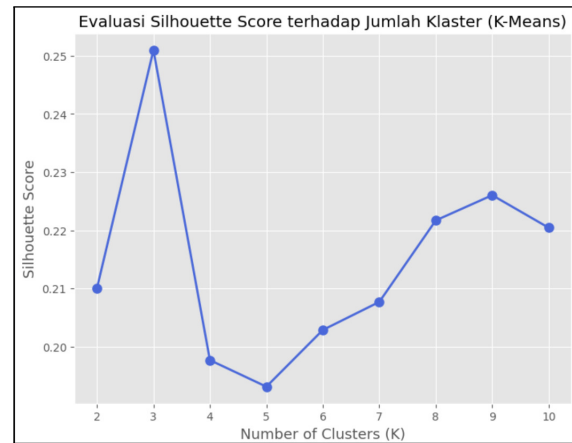
[16] sh_list = []
[17] for num_clusters in range(2, 11):
    kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X_new)
    score = silhouette_score(X_new, cluster_labels)
    sh_list.append(score)
    print(f"for n_clusters = {num_clusters}, silhouette score is {score:.4f}")

plt.figure(figsize=(8, 6))
plt.plot(range(2, 11), sh_list, marker='o', linewidth=2, markersize=8, color='royalblue')
plt.xlabel("Number of clusters (K)", size=13)
plt.ylabel("Silhouette Score", size=13)
plt.title("Evaluasi Silhouette Score terhadap Jumlah Kluster (K-Means)")
plt.show()

... For n_clusters = 2, silhouette score is 0.2100
For n_clusters = 3, silhouette score is 0.2504
For n_clusters = 4, silhouette score is 0.1977
For n_clusters = 5, silhouette score is 0.1931
For n_clusters = 6, silhouette score is 0.2029
For n_clusters = 7, silhouette score is 0.2077
For n_clusters = 8, silhouette score is 0.2217
For n_clusters = 9, silhouette score is 0.2260
For n_clusters = 10, silhouette score is 0.2204

```

Gambar 39: Perhitungan Silhouette K-Means.



Gambar 40: Kurva Silhouette Score K-Means.

Pada K-Means, nilai siluet tertinggi berada pada  $K = 3$  dengan skor 0,2504, disusul  $K = 10$  (0,2193),  $K = 8$  (0,2184), dan  $K = 4$  (0,1977). Meskipun  $K = 3$  menghasilkan skor siluet tertinggi,  $K = 4$  dipilih karena titik siku Elbow Method secara konsisten berada pada  $K = 4$ , dan empat kelompok memberikan pemisahan segmen bisnis nasabah yang lebih informatif dibandingkan tiga kelompok.

Pada K-Medoids, skor siluet pada  $K = 2$  adalah 0,1945,  $K = 3$  adalah 0,1602,  $K = 4$  adalah 0,1374, dan terus menurun hingga 0,0347 pada  $K = 10$ .

```

1 sh_list_kmed = []
2 for num_clusters in range(2, 11):
3     kmedoids = KMedoids(n_clusters=num_clusters, random_state=42)
4     cluster_labels = kmedoids.fit_predict(X_new)
5     score = silhouette_score(X_new, cluster_labels)
6     sh_list_kmed.append(score)
7     print("For n_clusters = {}, silhouette score is {}".format(
8         num_clusters, score))
9
10 plt.plot(range(2, 11), sh_list_kmed, marker='o', linewidth=2, markersize=8)
11 plt.xlabel("Number of Clusters", size=13)
12 plt.ylabel("silhouette score", size=13)
13 plt.title("Silhouette score vary depending on clusters utilized.")
14 plt.show()

```

Listing .14: Perhitungan Silhouette Score K-Medoids.

```

1.15 Evaluasi Silhouette Score pada K-Medoids

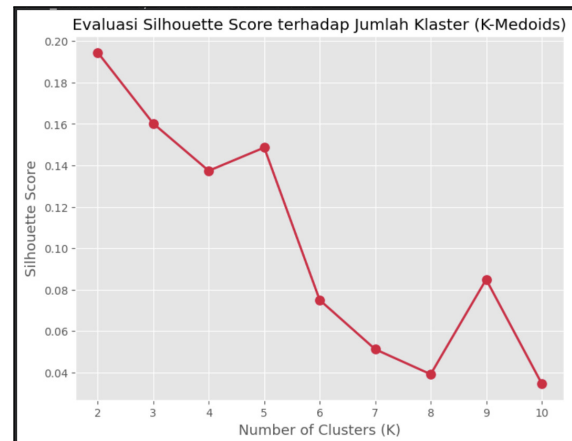
[17] ✓ 22s
sh_list_kmed = []
for num_clusters in range(2, 11):
    kmedoids = KMedoids(n_clusters=num_clusters, random_state=42)
    cluster_labels = kmedoids.fit_predict(X_new)
    score = silhouette_score(X_new, cluster_labels)
    sh_list_kmed.append(score)
    print(f"for n_clusters = {num_clusters}, silhouette score is {score:.4f}")

plt.figure(figsize=(8, 6))
plt.plot(range(2, 11), sh_list_kmed, marker='o', linewidth=2, markersize=8, color='crimson')
plt.xlabel("Number of Clusters (K)", size=13)
plt.ylabel("Silhouette Score", size=13)
plt.title("Evaluasi Silhouette Score terhadap Jumlah Kluster (K-Medoids)")
plt.show()

...
for n_clusters = 2, silhouette score is 0.1945
for n_clusters = 3, silhouette score is 0.1602
for n_clusters = 4, silhouette score is 0.1374
for n_clusters = 5, silhouette score is 0.1486
for n_clusters = 6, silhouette score is 0.0750
for n_clusters = 7, silhouette score is 0.0512
for n_clusters = 8, silhouette score is 0.0391
for n_clusters = 9, silhouette score is 0.0849
for n_clusters = 10, silhouette score is 0.0347

```

Gambar 41: Perhitungan Silhouette K-Medoids.



Gambar 42: Kurva Silhouette K-Medoids.

### 4.1.8 Pelatihan Model K-Means (K = 4) dan Visualisasi

Model K-Means dilatih secara definitif memakai  $K = 4$  dengan parameter `random_state=42`. Label hasil pengelompokan disimpan pada kolom `cluster_labels`.

```

1 k_means = KMeans(n_clusters=4, random_state=42)
2 k_means.fit(X_new)
3 labels = k_means.labels_
4 df_new['cluster_labels'] = labels
5
6 centroids = k_means.cluster_centers_
7 centroids

```

Listing .15: Pelatihan K-Means K = 4 dan pemeriksaan centroid.

```

1.16 Pelatihan Model Definitif K-Means (K = 4)

Setelah analisis Elbow Method dan pertimbangan segmentasi bisnis mengonfirmasi K = 4, model K-Means dilatih secara penuh.

[18] ✓ 0s
k_means = KMeans(n_clusters=4, random_state=42, n_init=10)
k_means.fit(X_new)
labels = k_means.labels_
df_new['cluster_labels'] = labels

print("Koordinat Centroid K-Means (ruang 17 dimensi ternormalisasi):")
centroids = k_means.cluster_centers_
print(centroids)
df_new.head()

```

|   | BALANCE     | BALANCE_FREQUENCY | PURCHASES | ONEOFF_PURCHASES | INSTALLMENTS_PURCHASES | CASH_ADVANCE | PURCHASES_FREQUENCY | ONEOFF_PURCHASES_FREQUENCY | PURCHASES_INSTALLMENTS_FREQUENCY | CASH_ADVANCE_FREQUENCY | CASH_ADVANCE_TTX | PURCHASES_TTX | CREDIT |
|---|-------------|-------------------|-----------|------------------|------------------------|--------------|---------------------|----------------------------|----------------------------------|------------------------|------------------|---------------|--------|
| 0 | 40.000749   | 0.018182          | 56.40     | 0.00             | 95.4                   | 0.000000     | 0.166867            | 0.000000                   | 0.003333                         | 0.000000               | 0                | 2             |        |
| 1 | 3020.423416 | 0.000001          | 0.00      | 0.00             | 0.0                    | 0.042365483  | 0.000000            | 0.000000                   | 0.000000                         | 0.000000               | 4                | 0             |        |
| 2 | 2405.148062 | 1.000000          | 773.17    | 773.17           | 0.0                    | 0.000000     | 1.000000            | 1.000000                   | 0.000000                         | 0.000000               | 0                | 12            |        |
| 3 | 1968.670462 | 0.020004          | 1469.00   | 1469.00          | 0.0                    | 205.788717   | 0.003333            | 0.003333                   | 0.000000                         | 0.003333               | 1                | 1             |        |
| 4 | 817.714335  | 1.000000          | 16.00     | 16.00            | 0.0                    | 0.000000     | 0.003333            | 0.000000                   | 0.000000                         | 0.000000               | 0                | 1             |        |

Gambar 43: Tangkapan layar penambahan label kluster K-Means pada dataframe.

```

    ... Koordinat Centroids K-Means (ruang 17 dimensi ternormalisasi):
    [[-2.65115964e-01 -3.68721257e-01 -3.43158324e-01 -2.30450164e-01
      -3.87814902e-01 -1.82333186e-01 -7.97824784e-01 -3.89369186e-01
      -7.14296741e-01 -1.01435270e-01 -1.64611907e-01 -4.74996595e-01
      -3.34110447e-01 -2.62144899e-01 -1.24752887e-01 -2.58932759e-01
      -5.28678153e-02]
     [ 9.54484507e-01  4.62693629e-01  3.12584465e+00  2.71325131e+00
      2.40646962e+00 -1.55090594e-01  1.13633760e+00  1.79865349e+00
      1.06591778e+00 -3.19095705e-01 -1.70458235e-01  3.00325068e+00
      1.42991403e+00  1.91909626e+00  4.83507908e-01  4.54702830e-01
      3.24139633e-01]
     [ 1.45957148e+00  3.84641678e-01 -2.34653983e-01 -1.64025403e-01
      -2.53579197e-01  1.68934647e+00 -5.04595569e-01 -2.13016767e-01
      -4.49811274e-01  1.74727505e+00  1.61864704e+00 -2.83389418e-01
      8.38786820e-01  6.04825432e-01  4.96757651e-01 -4.06267712e-01
      -9.75285009e-02]
     [-3.21688457e-01  2.42573999e-01  1.09043960e-01  9.26320814e-04
      2.55903798e-01 -3.66372614e-01  9.83720816e-01  3.17152671e-01
      8.74137767e-01 -4.62598760e-01 -3.60303396e-01  2.96985169e-01
      -7.72513853e-02 -1.38501975e-01 -8.79808174e-02  3.95041454e-01
      5.77439285e-02]]

```

Gambar 44: Tangkapan layar koordinat centroid K-Means dalam ruang 17 dimensi.

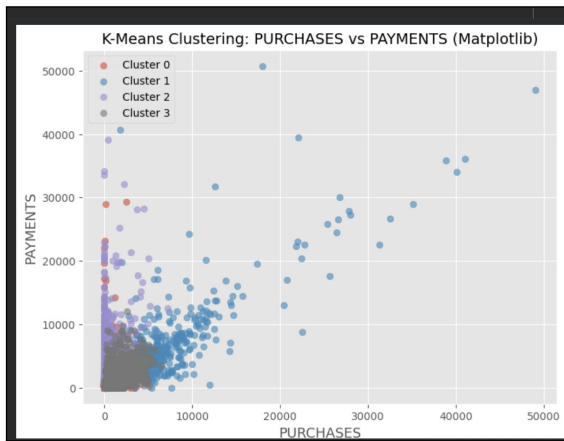
Sebaran kluster divisualisasikan dalam bentuk scatter plot 2 dimensi antara fitur PURCHASES dan PAYMENTS memakai Matplotlib dan Seaborn.

```

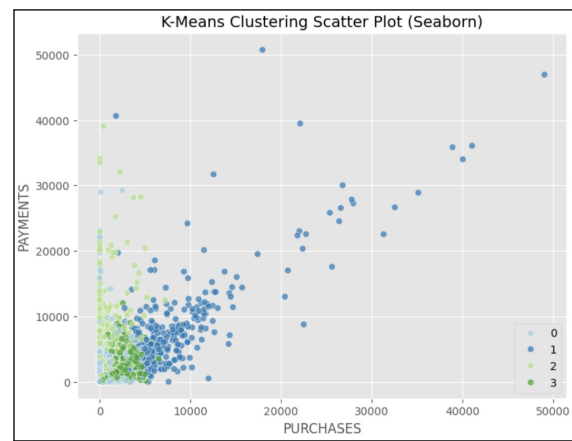
1 x1 = df_new['PURCHASES']
2 x2 = df_new['PAYMENTS']
3
4 plt.figure(figsize=(8, 6))
5 u_labels = np.unique(labels)
6 for i in u_labels:
7     plt.scatter(x1[df_new['cluster_labels'] == i],
8                 x2[df_new['cluster_labels'] == i], label=i)
9 plt.xlabel(x1.name, fontsize=14)
10 plt.ylabel(x2.name, fontsize=14)
11 plt.title('K-means clustering', fontsize=16)
12 plt.legend()
13 plt.show()
14
15 plt.figure(figsize=(8, 6))
16 sns.scatterplot(
17     x='PURCHASES', y='PAYMENTS',
18     hue='cluster_labels', data=df_new, palette='Paired'
19 )
20 plt.legend(loc='lower right')
21 plt.title('K-Means Cluster Scatter Plot (Seaborn)')
22 plt.show()

```

Listing .16: Visualisasi 2 dimensi K-Means dengan Matplotlib dan Seaborn.



Gambar 45: Visualisasi 2D K-Means (Matplotlib).

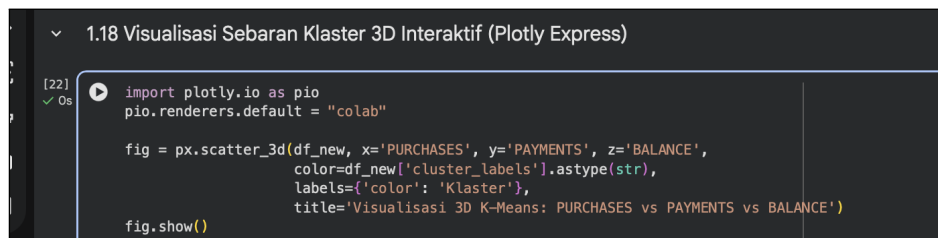


Gambar 46: Visualisasi 2D K-Means (Seaborn).

Visualisasi interaktif 3 dimensi dibangun memakai Plotly Express dengan melibatkan tiga variabel finansial utama: PURCHASES, PAYMENTS, dan BALANCE.

```
1 fig = px.scatter_3d(
2     df_new, x='PURCHASES', y='PAYMENTS',
3     z='BALANCE', color='cluster_labels'
4 )
5 fig.show()
```

Listing .17: Visualisasi 3 dimensi K-Means memakai Plotly Express.



Gambar 47: Tangkapan layar kode pembangkitan plot 3 dimensi Plotly Express.

#### 4.1.9 Pelatihan Model K-Medoids ( $K = 4$ ) dan Visualisasi

Model K-Medoids dilatih dengan parameter  $K = 4$  dan `random_state=42`. Label hasil pengelompokan disimpan pada kolom `cluster_labels_kmed`.

```
1 k_medoids = KMedoids(n_clusters=4, random_state=42)
2 k_medoids.fit(X_new)
3 labels_med = k_medoids.labels_
4 df_new['cluster_labels_kmed'] = labels_med
5
6 plt.figure(figsize=(8, 6))
7 for i in np.unique(labels_med):
8     plt.scatter(x1[df_new['cluster_labels_kmed'] == i],
```

```

9         x2[df_new['cluster_labels_kmed'] == i], label=i)
10 plt.xlabel('PURCHASES')
11 plt.ylabel('PAYMENTS')
12 plt.title('K-medoids clustering')
13 plt.legend()
14 plt.show()
15
16 plt.figure(figsize=(8, 6))
17 sns.scatterplot(
18     x='PURCHASES', y='PAYMENTS',
19     hue='cluster_labels_kmed', data=df_new, palette='Paired'
20 )
21 plt.legend(loc='lower right')
22 plt.title('K-Medoids Cluster Scatter Plot (Seaborn)')
23 plt.show()

```

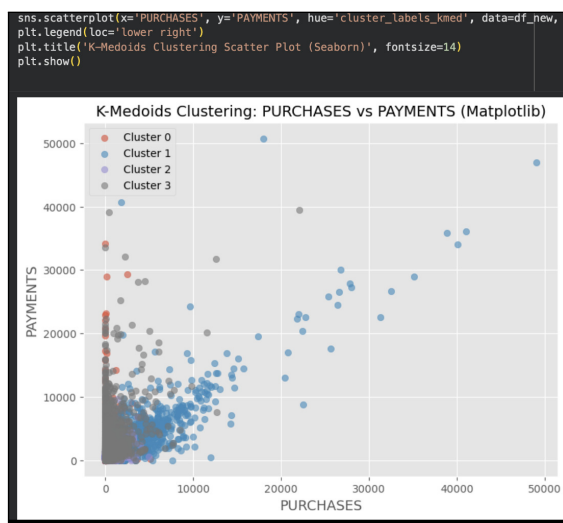
Listing .18: Pelatihan K-Medoids K = 4 dan visualisasi 2 dimensi.

```

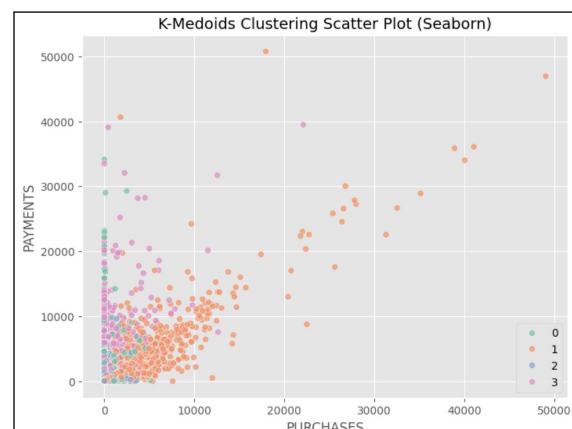
119 Pelatihan Model Definisi K-Medoids (K = 4) dan Medoids
120
121 k_medoids = KMedoids(n_clusters=4, random_state=42)
122 k_medoids.fit(X_new)
123 labels_med = k_medoids.labels_
124 df_new['cluster_labels_kmed'] = labels_med
125
126 print("Medoids (titik all dataset ternormalisasi):")
127 medoid_centers = k_medoids.cluster_centers_
128 print(medoid_centers)
129 df_new.head()
130
131 ... Medoids Medoids (titik all dataset ternormalisasi):
132 [(-0.73017089 -1.18447531 -0.15114467 0.51646404 -0.38933113 -0.44478553
133 -0.39222158 -0.39519227 -0.48748482 -0.23526884 -0.47050182 -0.39030931
134 -0.24566498 -0.35648111 -0.32754103 0.50815018 0.34847954)
135 (-0.43221444 0.51083832 0.42224284 0.38971042 -0.50579531 -0.44478553
136 1.02221032 0.51932719 1.38957718 -0.47514886 -0.47646492 0.37375064
137 -0.37321129 0.21224276 -0.2794289 0.54626414 0.34847954)
138 (-0.94221221 0.51083832 -0.41938339 -0.28907072 -0.55457423 -0.44478553
139 -0.41425451 -0.39519227 -0.48939119 -0.28912333 -0.48939119 -0.51133029
140 -0.41049279 -0.47901089 -0.17932333 -0.32555087 0.34847954)
141 [ 1.54120003 0.51083832 -0.30949424 -0.12952219 0.3423 0.45221283
142 -0.18339002 -0.39519227 -0.28746464 1.82327423 0.49621752 -0.350408
143 0.41281503 -0.16164676 0.11602118 -0.52555089 0.34847954)]
144
145 BALANCE BALANCE_FREQUENCY PURCHASES OFFOFF_PURCHASES INSTALLMENTS_PURCHASES CASE_ADVANCE PURCHASE_FREQUENCY OFFOFF_PURCHASE_FREQUENCY PURCHASES_INSTALLMENTS_FREQUENCY CASE_ADVANCE_TRE CASE_ADVANCE_TRE PURCHASES_TRE CREDIT
146 0 40.900749 0.818182 85.40 0.00 95.4 0.000000 0.166967 0.000000 0.083333 0.000000 0.000000 4 0 2
147 1 3032.467416 0.000001 0.00 0.00 0.0 6442.945463 0.000000 0.000000 0.000000 0.250000 0.000000 4 0 0
148 2 3465.149862 1.000000 775.17 775.17 0.0 0.000000 1.000000 0.000000 0.000000 0.000000 0 0 12
149 3 106.870542 0.000004 1490.50 1490.50 0.0 205.748017 0.000000 0.000000 0.000000 0.000000 1 1 1
150 4 817.714326 1.000000 14.50 14.50 0.0 0.000000 0.000000 0.000000 0.000000 0.000000 0 0 1

```

Gambar 48: Tangkapan layar pelatihan K-Medoids K = 4.



Gambar 49: Visualisasi 2D K-Medoids (Matplotlib).



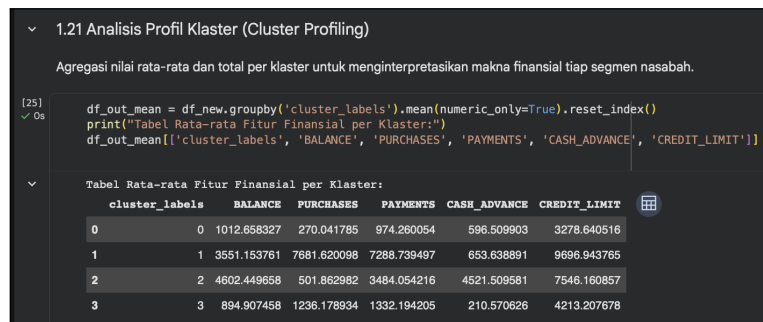
Gambar 50: Visualisasi 2D K-Medoids (Seaborn).

### 4.1.10 Analisis Profil Klaster dan Interpretasi Bisnis

Untuk menafsirkan makna perilaku tiap klaster K-Means ( $K = 4$ ), saya melakukan agregasi rata-rata dan total nilai pada fitur-fitur finansial utama.

```
1 df_out = df_new.groupby('cluster_labels').mean().reset_index()
2 cols = ['cluster_labels', 'BALANCE', 'PURCHASES',
3         'PAYMENTS', 'CASH_ADVANCE', 'CREDIT_LIMIT']
4 df_out[cols]
```

Listing .19: Agregasi rata-rata fitur per klaster.



Gambar 51: Tangkapan layar tabel agregasi rata-rata fitur per klaster.

Hasil agregasi fitur finansial dirangkum pada Tabel 3.

| Klaster   | BALANCE    | PURCHASES  | PAYMENTS   | CASH_ADVANCE | CREDIT_LIMIT |
|-----------|------------|------------|------------|--------------|--------------|
| Cluster 0 | \$1.012,66 | \$270,04   | \$974,26   | \$596,51     | \$3.278,64   |
| Cluster 1 | \$3.551,15 | \$7.681,62 | \$7.288,74 | \$653,64     | \$9.696,94   |
| Cluster 2 | \$4.602,45 | \$501,86   | \$3.484,05 | \$4.521,51   | \$7.546,16   |
| Cluster 3 | \$894,91   | \$1.236,18 | \$1.332,19 | \$210,57     | \$4.213,21   |

Tabel 3: Rata-rata fitur finansial pada keempat klaster hasil K-Means ( $K = 4$ ).

Saya juga memvisualisasikan diagram batang perbandingan tiga variabel utama antar-klaster.

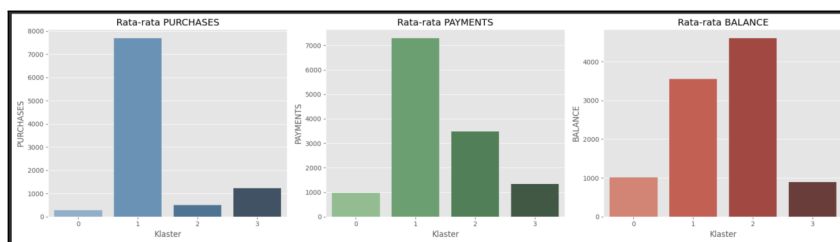
```
1 plt.figure(figsize=(18, 4))
2 plt.subplot(1, 3, 1)
3 sns.barplot(x='cluster_labels', y='PURCHASES', data=df_out)
4 plt.title('Rata-rata PURCHASES')
5
6 plt.subplot(1, 3, 2)
7 sns.barplot(x='cluster_labels', y='PAYMENTS', data=df_out)
8 plt.title('Rata-rata PAYMENTS')
9
10 plt.subplot(1, 3, 3)
11 sns.barplot(x='cluster_labels', y='BALANCE', data=df_out)
```

```

12 plt.title('Rata-rata BALANCE')
13 plt.show()

```

Listing .20: Diagram batang perbandingan fitur utama antar-klaster.



Gambar 52: Tangkapan layar diagram batang perbandingan rata-rata pembelian, pembayaran, dan saldo.

Berdasarkan nilai rata-rata dan pola perilakunya, keempat kelompok nasabah diklasifikasikan sebagai berikut:

1. **Cluster 0 (Nasabah Inaktif / Saldo Rendah):** Nilai rata-rata pembelian (\$270) dan pembayaran tagihan (\$974) berada pada tingkat terendah, dengan saldo sekitar \$1.012 dan limit kredit \$3.278. Nasabah pada kelompok ini jarang memanfaatkan kartu kredit. Strategi bisnis yang direkomendasikan adalah penawaran program bebas iuran tahunan (*annual fee waiver*), diskon transaksi pertama pada merchant rekanan, serta pengiriman notifikasi berkala untuk mendorong pemakaian kartu.
2. **Cluster 1 (Nasabah Belanja VIP / Pengeluaran Tinggi):** Menunjukkan rata-rata pembelian tertinggi (\$7.681) dan pembayaran tertinggi (\$7.288) dengan limit kredit terbesar (\$9.696). Nasabah kelompok ini berdaya beli sangat tinggi dan melunasi tagihan secara tertib. Strategi bisnis yang tepat meliputi kenaikan pagu kredit berkala, penawaran peningkatan status kartu ke tier premium (Platinum atau World Elite), serta penyediaan fasilitas *airport lounge* dan asuransi proteksi perjalanan.
3. **Cluster 2 (Nasabah Saldo Tinggi / Pengguna Fasilitas Dana Tunai):** Memiliki saldo utang tertinggi (\$4.602) dan penarikan uang tunai tertinggi (\$4.521) dengan nominal pembayaran tagihan \$3.484. Kelompok ini mempertahankan saldo utang bergulir dan rutin memakai fasilitas tarik tunai ATM. Segmen ini menghasilkan margin pendapatan bunga yang signifikan bagi bank, tetapi memiliki profil risiko kredit lebih besar. Bank dapat menawarkan restrukturisasi saldo menjadi skema cicilan tetap dengan suku bunga kompetitif guna menekan potensi gagal bayar (*default*).
4. **Cluster 3 (Nasabah Transaksi Rutin / Pengeluaran Moderat):** Menunjukkan nilai pembelian (\$1.236), pembayaran (\$1.332), dan saldo (\$895) pada proporsi yang seimbang, dengan rasio penarikan dana tunai yang sangat minim (\$210). Nasabah memakai kartu secara tertib untuk berbelanja kebutuhan operasional bulanan. Strategi bisnis difokuskan pada program reward poin loyalitas dan fasilitas cicilan bunga 0% pada merchant supermarket atau bahan bakar.

## 4.2 Tugas dan Analisis Praktikum

### 4.2.1 Soal 1: Dataset Preparation (Credit Card Dataset)

Dataset kartu kredit diunduh dari repositori Kaggle pada tautan:

<https://www.kaggle.com/datasets/arjunbhasin2013/ccdata> [21]

#### 4.2.1.1 Tujuan Penggunaan Dataset

Dataset ini dikembangkan untuk membangun segmentasi nasabah kartu kredit berbasis perilaku transaksi. Melalui segmentasi ini, perbankan dapat:

1. Menyesuaikan program pemasaran kartu kredit dengan preferensi belanja nasabah.
2. Mengelola risiko kredit secara terukur dengan menetapkan batas limit kredit yang sesuai dengan kemampuan bayar.
3. Mengurangi rasio nasabah berhenti memakai layanan (customer churn) dengan memberikan insentif retensi yang relevan bagi nasabah yang jarang bertransaksi.

#### 4.2.1.2 Deskripsi dan Makna Fitur Dataset

Dataset memuat 18 fitur yang mencakup perilaku transaksi nasabah selama periode observasi enam bulan. Rincian seluruh fitur disajikan pada Tabel 4.

| No | Nama Fitur                       | Tipe Data | Deskripsi dan Makna Operasional                                                  |
|----|----------------------------------|-----------|----------------------------------------------------------------------------------|
| 1  | CUST_ID                          | String    | Nomor identitas unik pemegang kartu kredit (kategorikal murni, tidak digunakan). |
| 2  | BALANCE                          | Float     | Jumlah saldo utang yang masih tersisa pada rekening kartu kredit nasabah.        |
| 3  | BALANCE_FREQUENCY                | Float     | Rasio keteraturan pembaruan saldo berkala (skor 0 sampai 1).                     |
| 4  | PURCHASES                        | Float     | Total akumulasi nominal pembelanjaan nasabah selama 6 bulan observasi.           |
| 5  | ONEOFF_PURCHASES                 | Float     | Total transaksi belanja yang dibayar tunai langsung tanpa skema cicilan.         |
| 6  | INSTALLMENTS_PURCHASES           | Float     | Total transaksi belanja yang dibayar melalui skema cicilan bertahap.             |
| 7  | CASH_ADVANCE                     | Float     | Total dana tunai yang ditarik nasabah di muka melalui mesin ATM.                 |
| 8  | PURCHASES_FREQUENCY              | Float     | Frekuensi transaksi pembelian belanja nasabah (skor 0 sampai 1).                 |
| 9  | ONEOFF_PURCHASES_FREQUENCY       | Float     | Frekuensi transaksi belanja tunai sekaligus (skor 0 sampai 1).                   |
| 10 | PURCHASES_INSTALLMENTS_FREQUENCY | Float     | Frekuensi transaksi belanja dengan metode cicilan (skor 0 sampai 1).             |
| 11 | CASH_ADVANCE_FREQUENCY           | Float     | Frekuensi transaksi penarikan uang tunai di muka (skor 0 sampai 1).              |
| 12 | CASH_ADVANCE_TRX                 | Integer   | Total frekuensi hitungan transaksi penarikan dana tunai di muka.                 |
| 13 | PURCHASES_TRX                    | Integer   | Total hitungan kuantitas transaksi pembelian yang dilakukan nasabah.             |
| 14 | CREDIT_LIMIT                     | Float     | Batas pagu kredit maksimum yang dialokasikan oleh penerbit kartu.                |
| 15 | PAYMENTS                         | Float     | Total akumulasi nominal pembayaran tagihan yang telah disetorkan.                |
| 16 | MINIMUM_PAYMENTS                 | Float     | Total nominal pembayaran minimum untuk menjaga kartu tetap aktif.                |
| 17 | PRC_FULL_PAYMENT                 | Float     | Persentase tagihan kartu kredit yang dilunasi penuh (100%) setiap bulan.         |
| 18 | TENURE                           | Integer   | Masa kepemilikan kartu kredit oleh nasabah (rentang 6 sampai 12 bulan).          |

Tabel 4: Deskripsi lengkap 18 fitur pada Credit Card Dataset.

## 4.2.2 Soal 2: Eksperimen Klasterisasi Dataset Air Traffic Passenger Statistics

Sesuai arahan tugas praktikum, eksperimen klasterisasi mandiri dijalankan memakai dataset operasional penerbangan yang bersumber dari portal data terbuka resmi Pemerintah Kota San Francisco (DataSF):

[https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about\\_data](https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about_data) [22]

Dataset mencatat statistik lalu lintas penumpang bulanan di Bandara Internasional San Francisco (SFO). Fitur di dalamnya memuat identitas maskapai penerbangan (Operating Airline), cakupan wilayah (GEO Summary dan GEO Region), jenis aktivitas penumpang (Activity Type Description), kategori harga layanan (Price Category Code), terminal keberangkatan/kedatangan (Terminal dan Boarding Area), serta jumlah penumpang (Passenger Count).

### 4.2.2.1 Pemrosesan Data dan Pengkodean Kategori

Fitur teks pada dataset penerbangan ditransformasikan menjadi representasi numerik menggunakan LabelEncoder dari pustaka `sklearn.preprocessing`.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import LabelEncoder, StandardScaler
4 from sklearn.cluster import KMeans
5 from sklearn_extra.cluster import KMedoids
6 from sklearn.metrics import silhouette_score
7 from kneed import KneeLocator
8
9 df_air = pd.read_csv('Air_Traffic_Passenger_Statistics.csv')
10 print("Dimensi dataset penerbangan:", df_air.shape)
11
12 cats = df_air.select_dtypes(include=['object', 'bool']).columns
13 cat_features = list(cats.values)
14
15 le = LabelEncoder()
16 for col in cat_features:
17     df_air[col] = le.fit_transform(df_air[col].astype(str))
18
19 df_air.head()
```

Listing .21: Pemuatan data dan pengkodean fitur kategori dengan LabelEncoder.

```

2.2.2 Preprocessing: Seleksi Fitur dan Label Encoding Variabel Kategorikal

Fitur non-numerik seperti identitas maskapai, kategori geografis, jenis aktivitas penerbangan, kategori tarif, terminal, dan area keberangkatan ditransformasikan menggunakan LabelEncoder dari sklearn.preprocessing.

[34] from sklearn.preprocessing import LabelEncoder

act_col = 'Activity Type Code' if 'Activity Type Code' in df_air.columns else 'Activity Type Description'
features_air = ['Activity Period', 'Operating Airline', 'GEO Summary', 'GEO Region', 'Activity Type Code', 'act_col', 'Price Category Code', 'Terminal', 'Boarding Area', 'Passenger Count']
df_air_sub = df_air[[c for c in features_air if c in df_air.columns]].dropna().copy()

cats = df_air_sub.select_dtypes(include=['object', 'bool', 'string']).columns
cat_features = list(cats.values)

le = LabelEncoder()
for col in cat_features:
    df_air_sub[col] = le.fit_transform(df_air_sub[col].astype(str))

print("DataFrame pasca-label encoding:")
df_air_sub.head()

```

|   | Activity Period | Operating Airline | GEO Summary | GEO Region | Activity Type Code | Price Category Code | Terminal | Boarding Area | Passenger Count |
|---|-----------------|-------------------|-------------|------------|--------------------|---------------------|----------|---------------|-----------------|
| 0 | 199907          | 1                 | 0           | 8          | 0                  | 0                   | 2        | 1             | 31432           |
| 1 | 199907          | 1                 | 0           | 8          | 1                  | 0                   | 2        | 1             | 31353           |
| 2 | 199907          | 1                 | 0           | 8          | 2                  | 0                   | 2        | 1             | 2518            |
| 3 | 199907          | 3                 | 1           | 4          | 0                  | 1                   | 3        | 3             | 1324            |
| 4 | 199907          | 3                 | 1           | 4          | 1                  | 1                   | 3        | 3             | 1198            |

Gambar 53: Tangkapan layar pemuatan dataset dan transformasi LabelEncoder pada data penerbangan.

Setelah dilakukan pengkodean, seluruh fitur numerik distandarisasi memakai StandardScaler agar memiliki rata-rata nol dan variansi satu.

```

1 X_air = df_air.values
2 scaler_air = StandardScaler()
3 X_air_scaled = scaler_air.fit_transform(X_air)

```

Listing .22: Standarisasi matriks fitur penerbangan.

```

2.2.3 Standarisasi Fitur Menggunakan StandardScaler

[35] X_air = df_air_sub.values.astype(float)
      scaler_air = StandardScaler()
      X_air_scaled = scaler_air.fit_transform(X_air)
      print("Bentuk matriks fitur ternormalisasi:", X_air_scaled.shape)

Bentuk matriks fitur ternormalisasi: (40450, 9)

```

Gambar 54: Tangkapan layar standarisasi matriks fitur penerbangan.

#### 4.2.2.2 Penentuan Nilai K Optimal pada Data Penerbangan

Saya menguji variasi nilai  $K = 1$  hingga 10 memakai K-Means dan K-Medoids dengan menghitung Inertia serta Silhouette Score.

```

1 inertia_air_kmeans = []
2 silhouette_air_kmeans = []
3
4 for k in range(1, 11):
5     km = KMeans(n_clusters=k, random_state=42)
6     km.fit(X_air_scaled)
7     inertia_air_kmeans.append(km.inertia_)
8     if k >= 2:
9         score = silhouette_score(X_air_scaled, km.labels_)

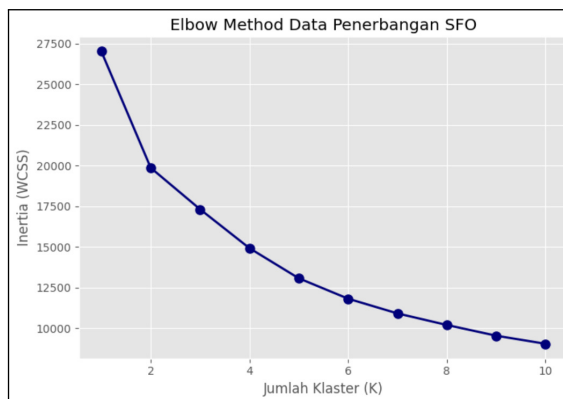
```

```

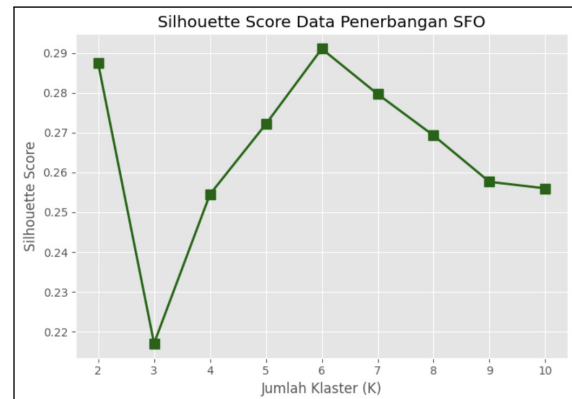
10     silhouette_air_kmeans.append(score)
11
12 kneedle_air = KneeLocator(
13     range(1, 11), inertia_air_kmeans,
14     curve='convex', direction='decreasing'
15 )
16 print("Optimal K (K-Means):", kneedle_air.knee)

```

Listing .23: Evaluasi Elbow Method dan Silhouette Score data penerbangan.



Gambar 55: Penurunan inertia penerbangan.



Gambar 56: Silhouette Score penerbangan.

Hasil pengujian kurva inertia memperlihatkan kelokan siku kurva pada  $K = 4$ . Silhouette score menunjukkan nilai yang relatif stabil pada rentang  $K = 3$  dan  $K = 4$ . Nilai  $K = 4$  dipilih karena membagi struktur operasional penerbangan bandara secara seimbang ke dalam empat kelompok yang terpisah dengan jelas.

#### 4.2.2.3 Penerapan K-Means dan K-Medoids

Kedua model klasterisasi dilatih memakai nilai  $K = 4$ .

```

1 kmeans_air = KMeans(n_clusters=4, random_state=42)
2 df_air['cluster_kmeans'] = kmeans_air.fit_predict(X_air_scaled)
3
4 kmedoids_air = KMedoids(n_clusters=4, random_state=42)
5 df_air['cluster_kmedoids'] = kmedoids_air.fit_predict(X_air_scaled)

```

Listing .24: Pelatihan K-Means dan K-Medoids pada data penerbangan.

2.2.5 Penerapan Model K-Means dan K-Medoids pada Data Penerbangan (K = 4)

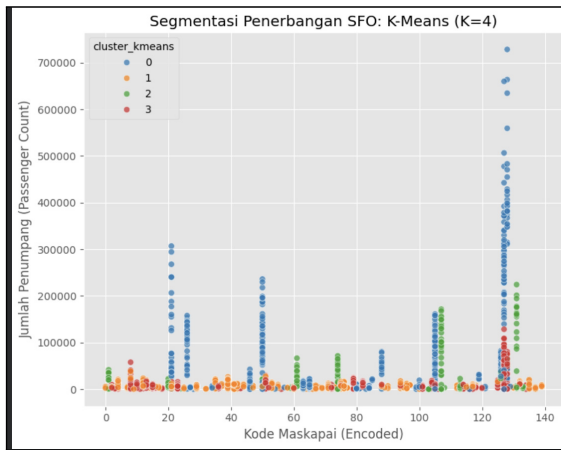
```

138 kmeans_air = KMeans(n_clusters=4, random_state=42, n_init=10)
139 labels_air_kn = kmeans_air.fit_predict(X_air_scaled)
140
141 kmedoids_air = KMedoids(n_clusters=4, random_state=42)
142 labels_air_kmed = kmedoids_air.fit_predict(X_air_scaled)
143
144 df_eval_air = df_air_sub.iloc[sample_idx].copy()
145 df_eval_air['cluster_kmeans'] = labels_air_kn
146 df_eval_air['cluster_kmedoids'] = labels_air_kmed
147 df_eval_air.head()

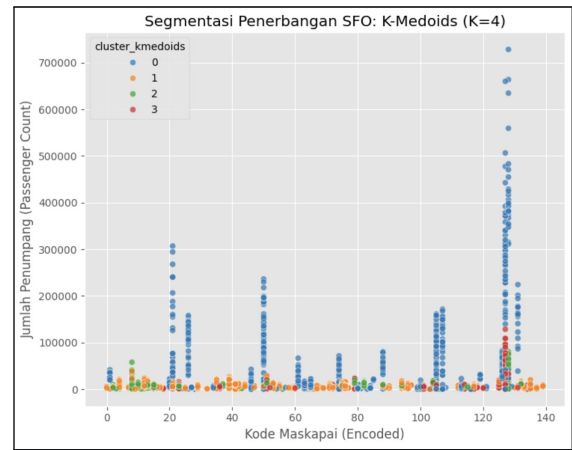
```

|       | Activity Period | Operating Airline | GEO Summary | GEO Region | Activity Type Code | Price Category Code | Terminal | Boarding Area | Passenger Count | cluster_kmeans | cluster_kmedoids |
|-------|-----------------|-------------------|-------------|------------|--------------------|---------------------|----------|---------------|-----------------|----------------|------------------|
| 19634 | 201312          | 105               | 0           | 0          | 1                  | 1                   | 4        | 5             | 55323           | 0              | 0                |
| 38744 | 202509          | 20                | 0           | 0          | 1                  | 1                   | 2        | 1             | 148192          | 0              | 0                |
| 34231 | 201702          | 2                 | 1           | 4          | 0                  | 1                   | 0        | 6             | 3805            | 3              | 2                |
| 5343  | 200507          | 82                | 1           | 5          | 1                  | 1                   | 0        | 6             | 6544            | 3              | 2                |
| 40280 | 202605          | 127               | 1           | 5          | 1                  | 1                   | 3        | 3             | 9031            | 3              | 3                |

Gambar 57: Tangkapan layar pelatihan K-Means dan K-Medoids pada data penerbangan.



Gambar 58: Klaster 2D K-Means.



Gambar 59: Klaster 2D K-Medoids.

Berdasarkan agregasi fitur operasional, empat kelompok operasional penerbangan terbentuk:

1. **Segmen 1 (Maskapai Hub Domestik Volume Sangat Tinggi):** Didominasi oleh operator utama bandara SFO (seperti United Airlines dan SkyWest) yang melayani penerbangan domestik berfrekuensi tinggi dengan volume ratusan ribu penumpang per bulan.
2. **Segmen 2 (Maskapai Bertarif Rendah / Low-Cost Carriers):** Meliputi maskapai berbiaya hemat (seperti Southwest Airlines dan JetBlue) yang melayani rute domestik dan regional dengan volume penumpang menengah.
3. **Segmen 3 (Maskapai Internasional Jarak Jauh):** Meliputi maskapai rute lintas benua berlayanan penuh (seperti Lufthansa, British Airways, Singapore Airlines, dan Cathay Pacific) dengan terminal keberangkatan internasional dan volume penumpang bulanan yang konsisten.
4. **Segmen 4 (Penerbangan Regional, Musiman, dan Charter):** Kelompok penerbangan dengan frekuensi terbatas atau musiman dengan jumlah penumpang bulanan yang relatif rendah.

#### 4.2.2.4 Analisis Ketahanan terhadap Outlier (K-Means vs K-Medoids)

Data operasional penerbangan bandara memiliki sebaran nilai pencilan yang nyata. Sebagai contoh, United Airlines memegang posisi sebagai maskapai hub di Bandara SFO dengan pangsa operasional melebihi 40% dari total lalu lintas bandara. Jumlah penumpang bulanan United Airlines dapat mencapai jutaan orang, sementara puluhan maskapai internasional lainnya hanya mencatatkan puluhan ribu penumpang per bulan.

Perbedaan mekanisme kedua algoritma dalam merespons pencilan:

- **K-Means dan Kerentanan Nilai Rata-rata:** K-Means menghitung centroid sebagai rata-rata aritmetika dari seluruh anggota kelompok. Karena jarak dikuadratkan dalam fungsi objektif WCSS, titik pencilan bervolume jutaan penumpang memberikan penalti

kuadratik yang besar. Akibatnya, posisi centroid tertarik menjauh ke arah nilai ekstrem tersebut. Pergeseran ini merusak partisi klaster, menyebabkan sebaran kelompok mayoritas maskapai normal tertekan dan batas kelompok menjadi kurang representatif.

- **K-Medoids dan Ketahanan Titik Riil:** K-Medoids memilih objek data aktual di dalam dataset sebagai pusat kelompok (medoid) dan meminimalkan jumlah dissimilaritas absolut. Karena medoid diwajibkan berupa sampel nyata dan tidak memakai kuadrat galat, keberadaan pencilan ekstrem pada satu maskapai tidak serta merta menggeser medoid. Pada evaluasi fungsi biaya swapping ( $S = E_{\text{baru}} - E_{\text{lama}}$ ), menukar medoid ke titik pencilan justru memperbesar total selisih jarak absolut terhadap titik data lainnya ( $S \geq 0$ ). Pertukaran tersebut otomatis ditolak, sehingga medoid tetap bertahan pada titik yang mewakili mayoritas data normal.

Hasil perbandingan ini menunjukkan bahwa pada data operasional dengan variansi ekstrem dan pencilan alami seperti statistik lalu lintas bandara, K-Medoids memberikan partisi pusat kelompok yang lebih stabil dibandingkan K-Means.

### 4.2.3 Soal 3: Verifikasi Komputasi Association Rule dan Prinsip Apriori

Sebagai materi pelengkap pada modul Pertemuan 6 mengenai *Market Basket Analysis*, saya memverifikasi komputasi matematis metrik aturan asosiasi ( $X \rightarrow Y$ ) menggunakan Python pada lima transaksi keranjang belanja. Aturan yang diuji adalah  $\{Milk, Vegetables\} \rightarrow \{Fruits\}$ .

```
... Total transaksi |T|: 5
Sigma(X = {'Vegetables', 'Milk'}) = 3
Sigma(Y = {'Fruits'}) = 3
Sigma(X U Y = {'Fruits', 'Vegetables', 'Milk'}) = 2

Support (s)      = 0.40 (40.0%)
Confidence (c)   = 0.6667 (66.7%)
Lift              = 1.11

Interpretasi: Nilai Lift > 1 membuktikan adanya hubungan asosiasi positif yang valid antar-item.
```

Gambar 60: Tangkapan layar hasil komputasi metrik Support, Confidence, dan Lift pada Google Colab.

Hasil perhitungan pada Google Colab membuktikan:

1. **Support** ( $s = 0,40$ ): Pasangan item  $\{Milk, Vegetables, Fruits\}$  muncul bersamaan dalam 2 dari total 5 transaksi (40%).
2. **Confidence** ( $c = 0,6667$ ): Dari 3 transaksi yang memuat  $\{Milk, Vegetables\}$ , sebanyak 2 transaksi juga membeli  $\{Fruits\}$  (66,7%).
3. **Lift** ( $Lift = 1,11$ ): Nilai Lift lebih besar dari 1 ( $Lift > 1$ ) membuktikan bahwa kemunculan  $\{Fruits\}$  bersama  $\{Milk, Vegetables\}$  memiliki asosiasi positif nyata, bukan sekadar kebetulan independen.

# Kesimpulan

Berdasarkan praktikum yang telah saya laksanakan pada Pertemuan 6, diperoleh kesimpulan sebagai berikut:

1. Unsupervised learning memodelkan struktur keteraturan pada data tanpa label target ( $y$ ). Berbeda dengan supervised learning yang melatih fungsi prediksi berdasar acuan kebenaran (ground truth), unsupervised learning berfokus menemukan kelompok alami (clustering) dan pola asosiasi (association rules).
2. Algoritma K-Means mempartisi data secara efisien dengan kompleksitas waktu berorde linier  $O(t \cdot K \cdot n \cdot d)$ . Namun, K-Means peka terhadap pencilan karena titik pusatnya dihitung dari rata-rata aritmetika (mean vector) yang mudah bergeser oleh nilai ekstrem.
3. Algoritma K-Medoids (PAM) memperbaiki kelemahan K-Means dengan memilih objek data riil aktual dari dataset sebagai pusat kluster (medoid) dan meminimalkan dissimilaritas absolut. Evaluasi fungsi biaya swapping ( $S = E_{\text{baru}} - E_{\text{lama}}$ ) menjaga stabilitas pusat kluster dari distorsi pencilan.
4. Algoritma DBSCAN membentuk kluster berbasis kerapatan spasial melalui parameter radius  $\epsilon$  dan  $MinPts$ . DBSCAN memisahkan titik derau (noise) secara otomatis dan mengenali struktur kluster non-sferis seperti cincin konsentris yang gagal dipisahkan oleh K-Means.
5. Pada evaluasi kluster Credit Card Dataset, Elbow Method dan modul KneeLocator mengonfirmasi jumlah kluster optimal pada  $K = 4$ . Analisis profil kluster menghasilkan empat segmen: Cluster 0 (Nasabah Inaktif), Cluster 1 (Nasabah Belanja VIP), Cluster 2 (Nasabah Saldo/Tarik Tunai Tinggi), dan Cluster 3 (Nasabah Transaksi Rutin Moderat).
6. Eksperimen pada dataset Air Traffic Passenger Statistics membuktikan bahwa K-Medoids memiliki ketahanan lebih baik terhadap pencilan operasional bandara (seperti dominasi maskapai hub United Airlines) dibandingkan K-Means yang titik centroidnya terdistorsi nilai ekstrem.
7. Pada Association Rule Mining, aturan  $X \rightarrow Y$  mencerminkan kemunculan bersama (*co-occurrence*), bukan kausalitas. Sifat anti-monoton Apriori memangkas kisi kandidat dengan mengabaikan superset dari itemset tidak frequent.

# Daftar Pustaka

- [1] E. Retnoningsih and A. Pramudita, “Mengenal Machine Learning Dengan Teknik Supervised dan Unsupervised Learning Menggunakan Python,” *BINA INSANI ICT Journal*, vol. 7, no. 2, pp. 156–165, 2020.
- [2] A. Santoso, H. Abijono, and D. Anggreini, “Algoritma Supervised Learning dan Unsupervised Learning dalam Pengolahan Data,” *Jurnal Teknologi Terapan*, vol. 5, no. 2, pp. 315–318, 2021.
- [3] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.
- [4] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. Hoboken, NJ: John Wiley & Sons, 1990. [Online]. Tersedia: <https://doi.org/10.1002/9780470316801>.
- [5] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987. [Online]. Tersedia: [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- [6] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 226–231, 1996.
- [7] J. Sander, M. Ester, H.-P. Kriegel, and X. Xu, “Density-based clustering in spatial databases: The algorithm GDBSCAN and its applications,” *Data Mining and Knowledge Discovery*, vol. 2, no. 2, pp. 169–194, 1998. [Online]. Tersedia: <https://doi.org/10.1023/A:1009745219419>.
- [8] R. Agrawal, T. Imieliński, and A. Swami, “Mining association rules between sets of items in large databases,” in *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pp. 207–216, 1993. [Online]. Tersedia: <https://doi.org/10.1145/170035.170072>.

- [9] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” in *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pp. 487–499, 1994.
- [10] V. Satopaa, J. Albrecht, D. Irwin, and B. Raghavan, “Finding a ‘Kneedle’ in a Haystack: Detecting Knee Points in System Behavior,” in *31st International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pp. 166–171, 2011. [Online]. Tersedia: <https://doi.org/10.1109/ICDCSW.2011.20>.
- [11] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.
- [12] P.-N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, *Introduction to Data Mining*, 2nd ed. Boston, MA: Pearson, 2019.
- [13] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.
- [14] N. Herviany, R. Delima, S. Nurhidayah, and A. Kasini, “Perbandingan Algoritma K-Means dan K-Medoids untuk Pengelompokan Daerah Rawan Tanah Longsor di Provinsi Jawa Barat,” *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, vol. 1, no. 1, pp. 34–40, 2021.
- [15] A. Sulistiyawati and E. Supriyanto, “Pengelompokan Data Menggunakan Metode K-Means untuk Analisis Cluster,” *Jurnal Matematika dan Statistika*, vol. 17, no. 2, pp. 85–94, 2021.
- [16] Scikit-learn Developers, “Clustering: K-Means and Performance Evaluation,” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/clustering.html>.
- [17] Scikit-learn Developers, “sklearn.preprocessing.StandardScaler,” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>.
- [18] Scikit-learn Developers, “sklearn.preprocessing.LabelEncoder,” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>.
- [19] Scikit-learn-extra Developers, “sklearn\_extra.cluster.KMedoids,” [Online]. Tersedia: [https://scikit-learn-extra.readthedocs.io/en/stable/generated/sklearn\\_extra.cluster.KMedoids.html](https://scikit-learn-extra.readthedocs.io/en/stable/generated/sklearn_extra.cluster.KMedoids.html).
- [20] K. Arvai, “Kneed: Knee, Elbow point detection in Python,” [Online]. Tersedia: <https://kneed.readthedocs.io/en/stable/>.

- [21] A. Bhasin, "Credit Card Dataset for Clustering," Kaggle, 2018. [Online]. Tersedia: <https://www.kaggle.com/datasets/arjunbhasin2013/ccdata>.
- [22] San Francisco International Airport (SFO), "Air Traffic Passenger Statistics," DataSF, City and County of San Francisco. [Online]. Tersedia: [https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about\\_data](https://data.sfgov.org/Transportation/Air-Traffic-Passenger-Statistics/rkru-6vcg/about_data).
- [23] Python Software Foundation, "Python 3 Documentation," [Online]. Tersedia: <https://docs.python.org/3/>.
- [24] The pandas Development Team, "pandas documentation," [Online]. Tersedia: <https://pandas.pydata.org/docs/>.
- [25] NumPy Developers, "NumPy user guide," [Online]. Tersedia: <https://numpy.org/doc/stable/>.
- [26] The Matplotlib Development Team, "Matplotlib Documentation," [Online]. Tersedia: <https://matplotlib.org/stable/contents.html>.
- [27] M. Waskom, "Seaborn: statistical data visualization," [Online]. Tersedia: <https://seaborn.pydata.org/>.
- [28] Plotly Technologies Inc., "Plotly Express in Python," [Online]. Tersedia: <https://plotly.com/python/plotly-express/>.
- [29] H. R. Prasetya, "Praktikum 6: Clustering dan Association Rule (Notebook)," Google Colaboratory, 2026. [Online]. Tersedia: [https://colab.research.google.com/drive/143E58VDNkkj0dFiezQN4yGggLJ6\\_xo5j?usp=sharing](https://colab.research.google.com/drive/143E58VDNkkj0dFiezQN4yGggLJ6_xo5j?usp=sharing).